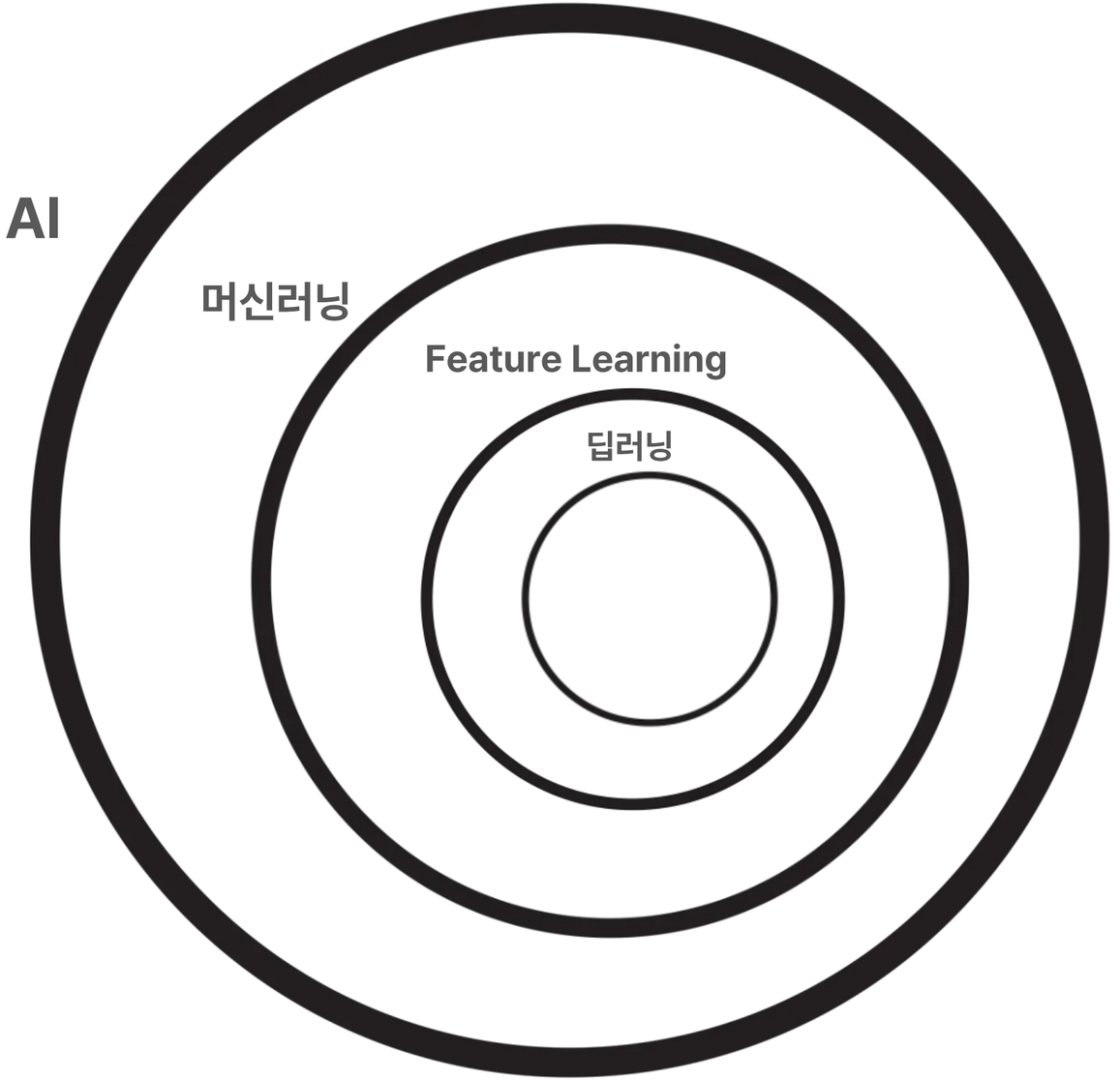
Winter AI Study
Week 1: Machine Learning

발표자 조현진

머신러닝 vs 딥러닝



머신러닝 vs 딥러닝

AI: 기계나 시스템에 지능이라는 개념을 도입

머신러닝: 데이터의 패턴을 파악해서 Task 수행하는 것의 전체 개념

Feature Learning: 데이터 내부의 feature를 직접 학습함으로써 데이터를 파악

딥러닝: feature learning의 구현 방법 중 하나 - 계산 층을 여러 개 쌓음

해당 개념들의 차이는 뒷 슬라이드로 천천히 이해하면 됩니다!

머신러닝 vs 딥러닝

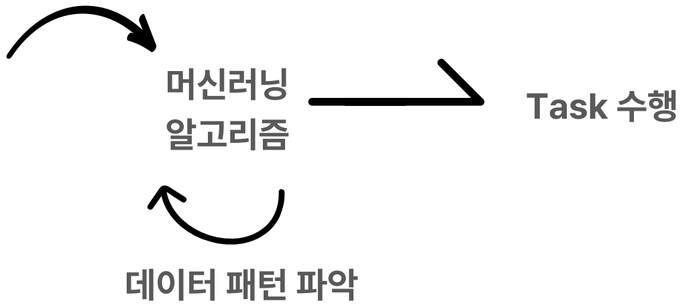
딥러닝이 아닌 머신러닝: 정형 데이터의 패턴을 파악

정형 데이터: 행과 열에 인간이 규정한 의미가 부여되어 있는 데이터

	Gene 1	Gene 2	Gene 3
Cell 1	0.5	0.7	0.2
Cell 2	0.4	0.2	0.1
Cell 3	0.7	0.9	0.6

Gene 1, 2, 3은 Cell을 표현하는 feature

인간이 직접 feature를 설정해놓아야 함 (딥러닝과의 핵심 차이)



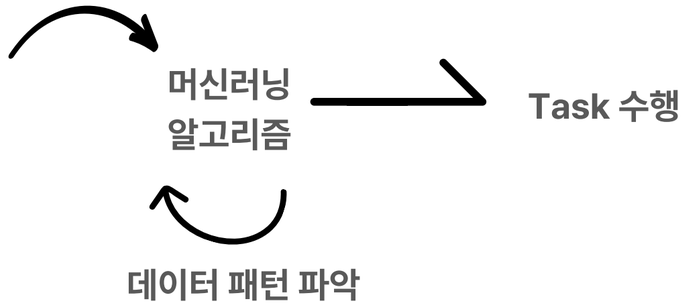
머신러닝 vs 딥러닝

딥러닝이 아닌 머신러닝: 정형 데이터의 패턴을 파악

정형 데이터: 행과 열에 인간이 규정한 의미가 부여되어 있는 데이터

	Feature 1	Feature 2	Feature 3
Sample 1			
Sample 2			
Sample 3			

데이터 샘플들을 설계한 feature의 조합으로 나타낸 표가 정형 데이터



머신러닝 vs 딥러닝

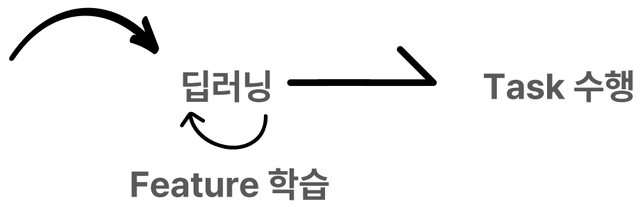
딥러닝: 비정형 데이터의 패턴도 파악 가능 (Feature Learning 기법)

Feature Learning?

모델이 데이터 내부에 내재되어 있는 feature들을 직접 뽑아내 학습 (!= 머신러닝은 feature를 직접 인간이 설계)

이미지 데이터: 픽셀이 행과 열을 이루고 있지만 각 픽셀에 우리가 의미를 부여하지는 않음(비정형 데이터 대표 1)





내부에서 데이터가 어떤 숫자들의 벡터 or 행렬로 표현됨 (우리는 이것들이 어떤 의미인지는 추상적으로만 해석 가능)

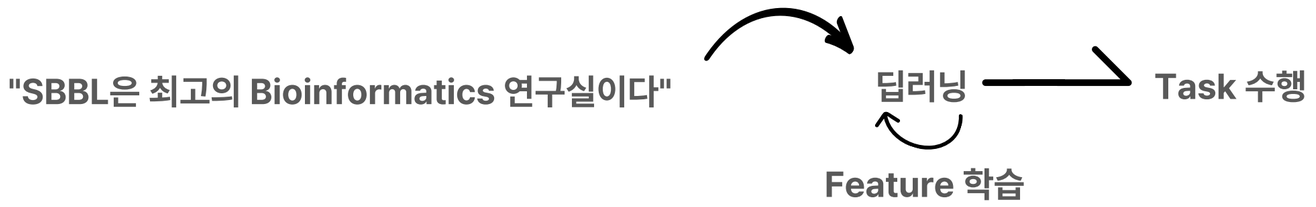
머신러닝 vs 딥러닝

딥러닝: 비정형 데이터의 패턴도 파악 가능 (Feature Learning 기법)

Feature Learning?

모델이 데이터 내부에 내재되어 있는 feature들을 직접 뽑아내 학습 (!= 머신러닝은 feature를 직접 인간이 설계)

자연어(Natural Language): 데이터가 애초에 행과 열이 아님 (비정형 데이터 대표 2)



내부에서 데이터가 어떤 숫자들의 벡터 or 행렬로 표현됨 (우리는 이것들이 어떤 의미인지는 추상적으로만 해석 가능)

머신러닝 vs 딥러닝

딥러닝: 비정형 데이터의 패턴도 파악 가능 (Feature Learning 기법)

Feature Learning?

모델이 데이터 내부에 내재되어 있는 feature들을 직접 뽑아내 학습 (!= 머신러닝은 feature를 직접 인간이 설계)

딥러닝 적용 분야 두 대표

Computer Vision (CV)
컴퓨터가 이미지를 이해하는 방식을 탐구하는 분야

Natural Language Processing (NLP, 자연어처리)
컴퓨터가 자연어를 이해하는 방식을 탐구하는 분야

머신러닝 vs 딥러닝

생물학 Data?

정형 데이터 대표 예시: scRNA-seq의 Cell x Gene matrix → 머신러닝, 딥러닝 둘 다 사용 가능

비정형 데이터 대표 예시: DNA sequence (AGGCTGAA....) → 딥러닝만 사용

머신러닝 vs 딥러닝(추가 슬라이드)

딥러닝이 아닌 머신러닝은 입력으로 준 데이터 값 자체만을 활용해 패턴을 파악함

반면, 딥러닝은 입력으로 준 데이터 값으로부터 다양한 층을 거치면서 우리가 이해하지 못하는 새로운 값들을
새로 만들어냄 → 학습이 잘 되면 결국 모델이 계산한 그 새로운 값들이 데이터 내부의 feature 정보를 잘 반영하는 값들이 되는 것임

이 점을 생각해보았을 때, 딥러닝이 아닌 머신러닝이 패턴 파악을 잘할 수 있는 데이터의 형태는 이미 잘 설계 되어 있는 형태여야 함을 이해할 수 있음
(열로 우리가 지정해준 feature들간의 상관관계가 우리가 구하고자 하는 예측 값이랑 어느정도 align이 잘 되어 있어야 함)
그래서 딥러닝이 아닌 머신러닝은 우리가 직접 feature를 잘 설계한 정형 데이터에 유리하다고 한 것!

일반적으로 행렬 형태면 딥러닝이든 머신러닝이든 동작은 잘 함 (성능 차이의 문제일 뿐)

학습이란? 목적이라는 것이 반드시 있어야 하고, 그 목적을 달성하기 위해 최적화 해나가는 과정



Objective Function (목적 함수)

목적을 수학적으로 정의 내린 것
목적 함수를 최대화 or 최소화하는 방향이 학습의 바람직한 방향

학습의 일반적인 과정

Task의 목적 함수를 수학적으로 설계

(공통된 목적 함수가 있다면 그걸 그대로 가지고 와서 사용해도 ok)

<목적 함수를 최대화 또는 최소화하는 인자 찾기>

argmax Obj func
or
argmin Obj func

목적 함수가 단순 미분으로 최대/최소 구하는 게 가능하다면 (Closed-form)

그냥 도함수 = 0의 해를 구해서 파라미터 값 계산

목적 함수가 closed-form이 아니라면, 파라미터 업데이트 한 번 해보고 결과 확인 → 또 업데이트 해보고 결과 확인 .... 이런 식으로 점진적 최적화

지도학습 vs 비지도학습

지도 학습: 학습 데이터에 정답지가 주어짐

비지도 학습: 학습 데이터에 정답지가 주어지지 않음

지도학습 vs 비지도학습

지도 학습: 목적 함수가 모델의 예측 값 뿐만 아니라 따로 주어지는 정답지 값으로도 변수 구성

비지도 학습: 목적 함수가 모델의 예측 값 또는 데이터 값으로만 변수 구성

지도학습 vs 비지도학습

지도 학습: 목적 함수가 데이터와 모델의 예측 값 뿐만 아니라 따로 주어지는 정답지 값으로도 변수 구성

정답 값 예측 값

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

예를 들어

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

지도학습 vs 비지도학습

비지도 학습: 목적 함수가 모델의 예측 값 또는 데이터 값으로 변수 구성 (따로 정답지를 넣어줄 필요 x)

예를 들어

$$J = \sum_{i=1}^{m} \sum_{k=1}^{m} w_{ik} \|x_i - c_k\|^2$$

W: 클러스터에 속해 있지 않은 점이면 0

x: 데이터 포인트

Ck: 클러스터 중심 (데이터로부터 계산, 따로 제공한 정답지가 아님)

$$w_{ik} = 0 \; if \; the \; data \; point \; does \; not \; belong \; to \; the \; cluster$$
$$w_{ik} = 1 \; if \; the \; data \; point \; belongs \; to \; the \; cluster$$

Clustering 목적 함수

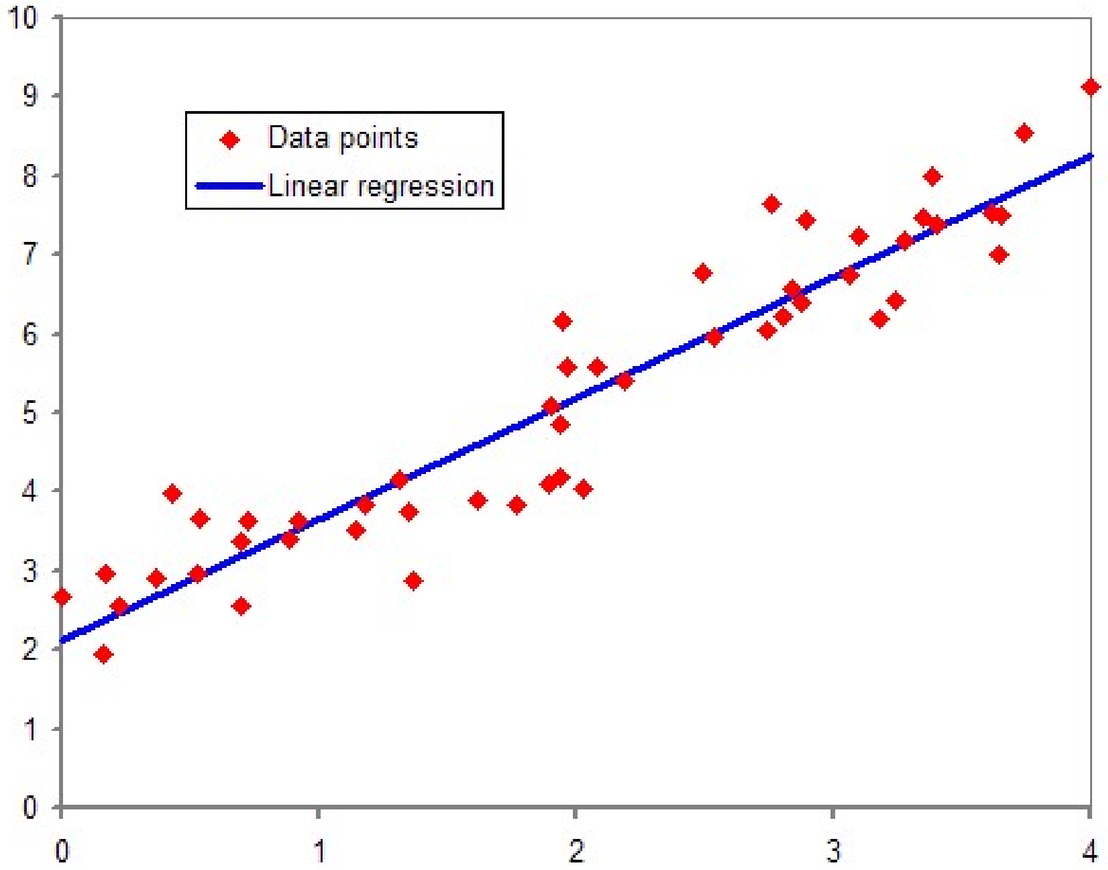
지도학습 vs 비지도학습

강화학습? (Reinforcement Learning)

모델이 환경에 action을 취할 때마다 환경으로부터 데이터를 취득(observation)하여
장기적 reward(보상)을 최대화 하는 방향으로 어떤 action을 해야 하는지 학습

Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)



회귀 머신러닝 알고리즘 1: LinearRegression

```
from sklearn.linear_model import LinearRegression
```

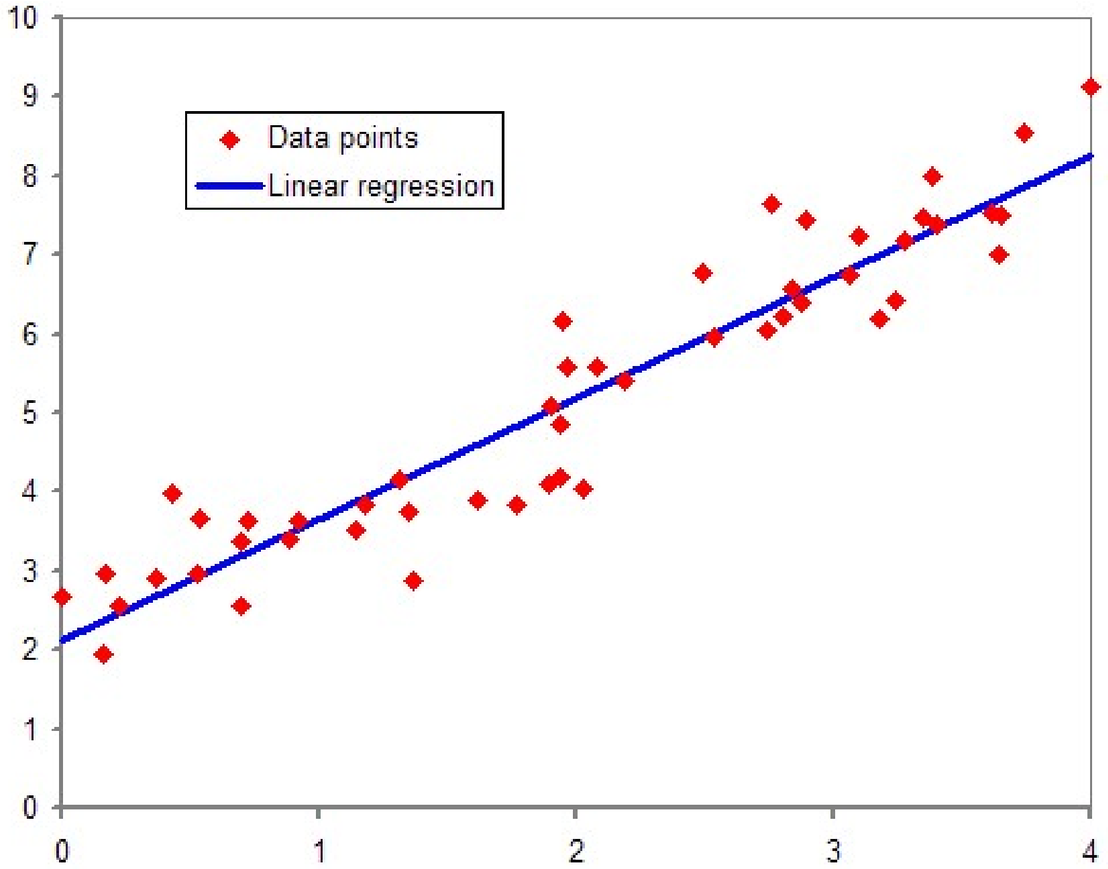
데이터의 분포를 단순 선으로 근사 예측

선 밖에 있는 데이터도 일단 선으로 회귀 시킨다고 해서 회귀라고 명명

$y = Wx + b$ (W, b는 학습 파라미터)

Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)



회귀 머신러닝 알고리즘 1: LinearRegression

목적 함수: MSE

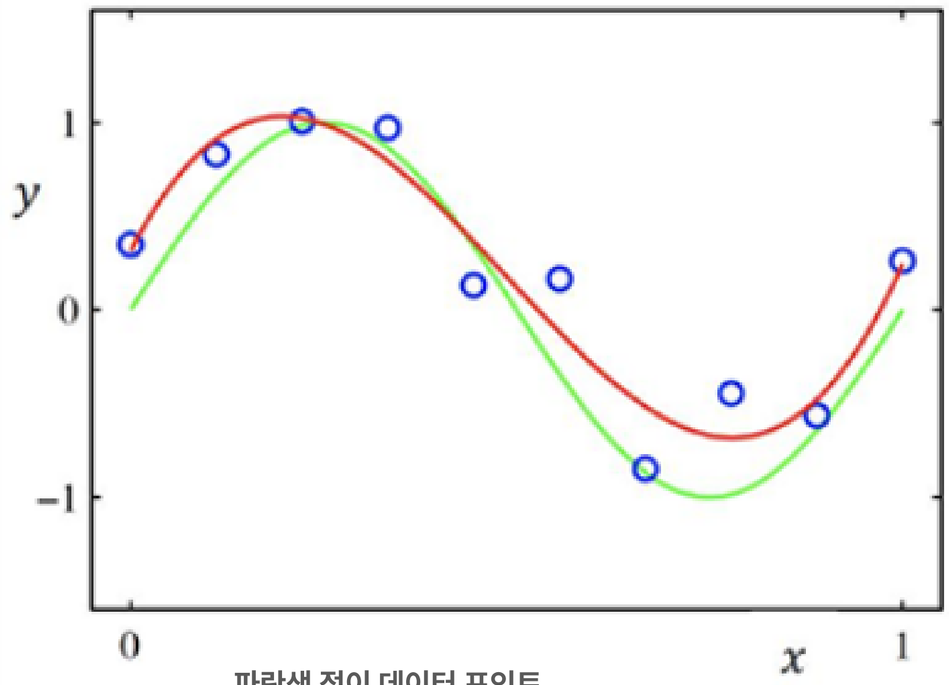
$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

MSE = mean squared error
n = number of data points
Y_i = observed values
$\hat{Y}_i$ = predicted values

단순 선형 회귀 + MSE는 closed-form
→ 도함수 = 0해서 바로 최적 W, b를 구할 수 있음

Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

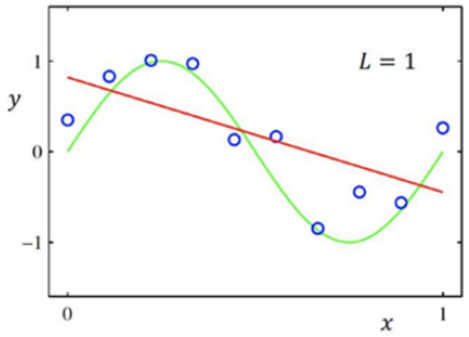
Regression: 수치 변수 예측 (즉, 값을 예측)



파란색 점이 데이터 포인트
초록색 선이 바람직한 회귀 곡선
빨간색 선이 계산된 회귀 곡선

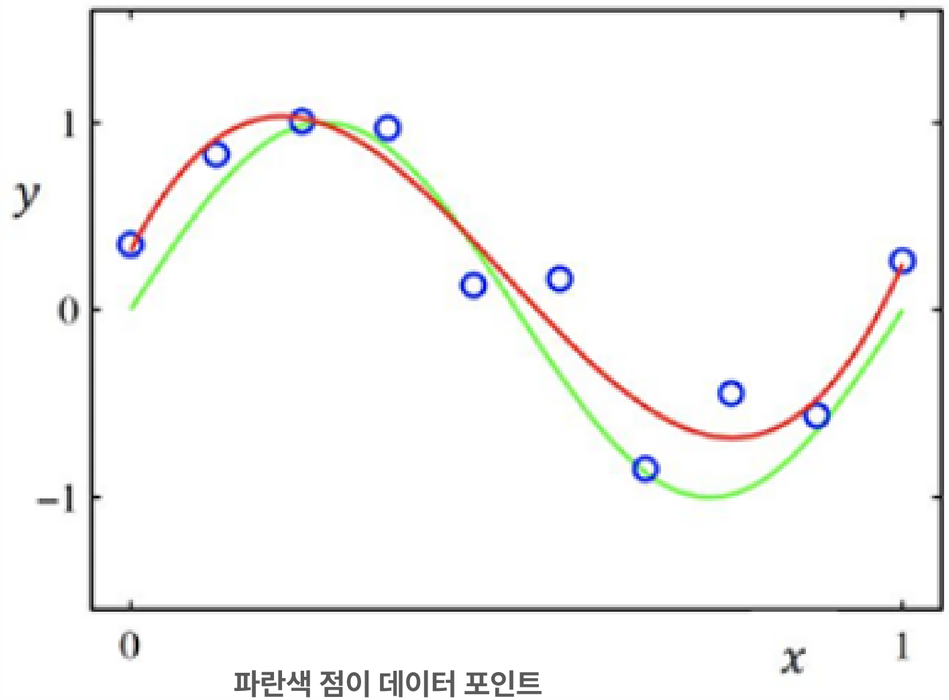
회귀 머신러닝 알고리즘 2: Polynomial Regression

어떤 데이터 분포는 직선으로 표현하기엔 복잡함
→ 곡선으로 표현해야 함



Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)



파란색 점이 데이터 포인트
초록색 선이 바람직한 회귀 곡선
빨간색 선이 계산된 회귀 곡선

회귀 머신러닝 알고리즘 2: Polynomial Regression

y = w0 + w1*x + w2*x^2 + w3*x^3 + ...

x: 입력 변수
w: 학습 파라미터

Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)

회귀 머신러닝 알고리즘 2: Polynomial Regression

$$y = w0 + w1*x + w2*x^2 + w3*x^3 + ...$$

Feature를 직접 생성

w0, w1, ... 계산 = 목적 함수 최적화

Feature 1개

x

```
from sklearn.preprocessing import PolynomialFeatures
```

x
x^2
x^3
...

```
from sklearn.linear_model import LinearRegression
```

Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)

회귀 머신러닝 알고리즘 3: Multiple Linear Regression

$$y = w1*x1 + w2*x2 + w3*x3 + ...$$

	Feature 1	Feature 2	Feature 3
Sample 1			
Sample 2			
Sample 3			

대부분의 정형 데이터는 입력 feature가 1개로만 구성되어 있지는 않음

Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)

회귀 머신러닝 알고리즘 3: Multiple Linear Regression

$$y = w1*x1 + w2*x2 + w3*x3 + ...$$

	x1	x2	x3
Sample 1			
Sample 2			
Sample 3			

w1 학습: 입력 feature x1에 가중치를 얼마나 둬야 하나 학습
w2 학습: 입력 feature x2에 가중치를 얼마나 둬야 하나 학습
....

대부분의 정형 데이터는 입력 feature가 1개로만 구성되어 있지는 않음

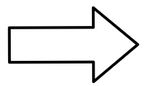
Regression(회귀) Task로 공부하는 머신러닝 (딥러닝 x)

Regression: 수치 변수 예측 (즉, 값을 예측)

회귀 머신러닝 알고리즘 3: Multiple Linear Regression

$$y = w1*x1 + w2*x2 + w3*x3 + ...$$

	x1	x2	x3
Sample 1			
Sample 2			
Sample 3			



w1, w2, ... 계산 = 목적 함수 최적화



Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

Classification: 샘플이 속한 class 예측



Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 1: KNN (K-nearest neighbor)

First step: 각 샘플의 feature들 수치 값을 벡터 원소로 하여 공간에 점을 찍음 (저장 단계 = 학습)

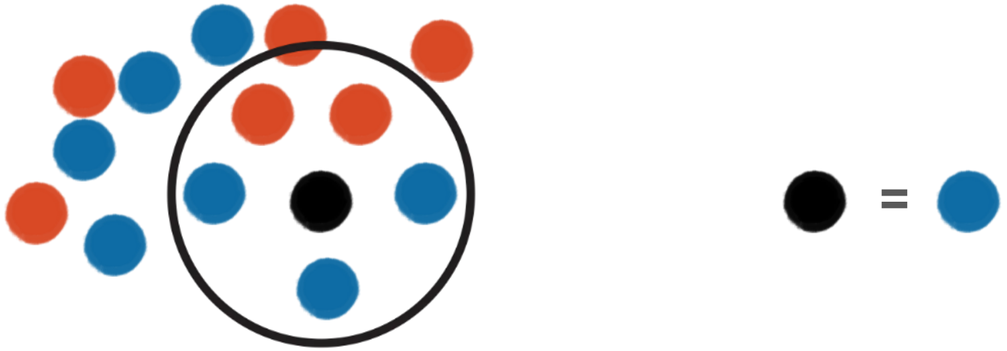


Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 1: KNN (K-nearest neighbor)

First step: 각 샘플을 feature들 수치 값을 벡터 원소로 하여 공간에 점을 찍음 (저장 단계=학습)

Second step: Inference할 때는 해당 점에서 가장 가까운 K개의 점을 뽑아 voting



Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 1: KNN (K-nearest neighbor)

First step: 각 샘플을 feature들 수치 값을 벡터 원소로 하여 공간에 점을 찍음 (저장=학습)

Second step: Inference할 때는 해당 점에서 가장 가까운 K개의 점을 뽑아 voting

좌표 공간 내에서 샘플 사이 가까운 지 판별하려면 거리 metrics(or 유사도)를 꼭 알아야 함!

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 1: KNN (K-nearest neighbor)

좌표 공간 내에서 샘플 사이 가까운 지 판별하려면 거리 metrics(or 유사도)를 꼭 알아야 함! (KNN이 아니더라도 중요함)

L1 Distance (Manhattan distance):

$$d_1(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^{n} |p_i - q_i|$$

L2 Distance (Euclidean distance):

$$d_2(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}$$

점 A = [2, 3, 4]
점 B = [1, 1, 2]

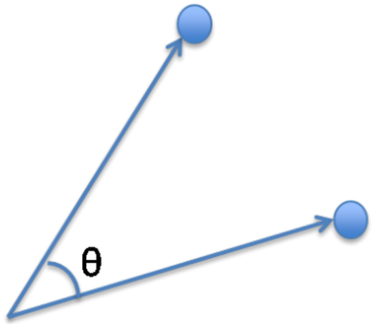
L1 dist = |2 - 1| + |3 - 1| + |4 - 2| = 5
L2 dist = ((2 - 1)^2 + (3 - 1)^2 + (4 - 2)^2)^0.5 = 3

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 1: KNN (K-nearest neighbor)

좌표 공간 내에서 샘플 사이 가까운 지 판별하려면 거리 metrics(or 유사도)를 꼭 알아야 함! (KNN이 아니더라도 중요함)

$$sim(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$



코사인 유사도(매우 중요)

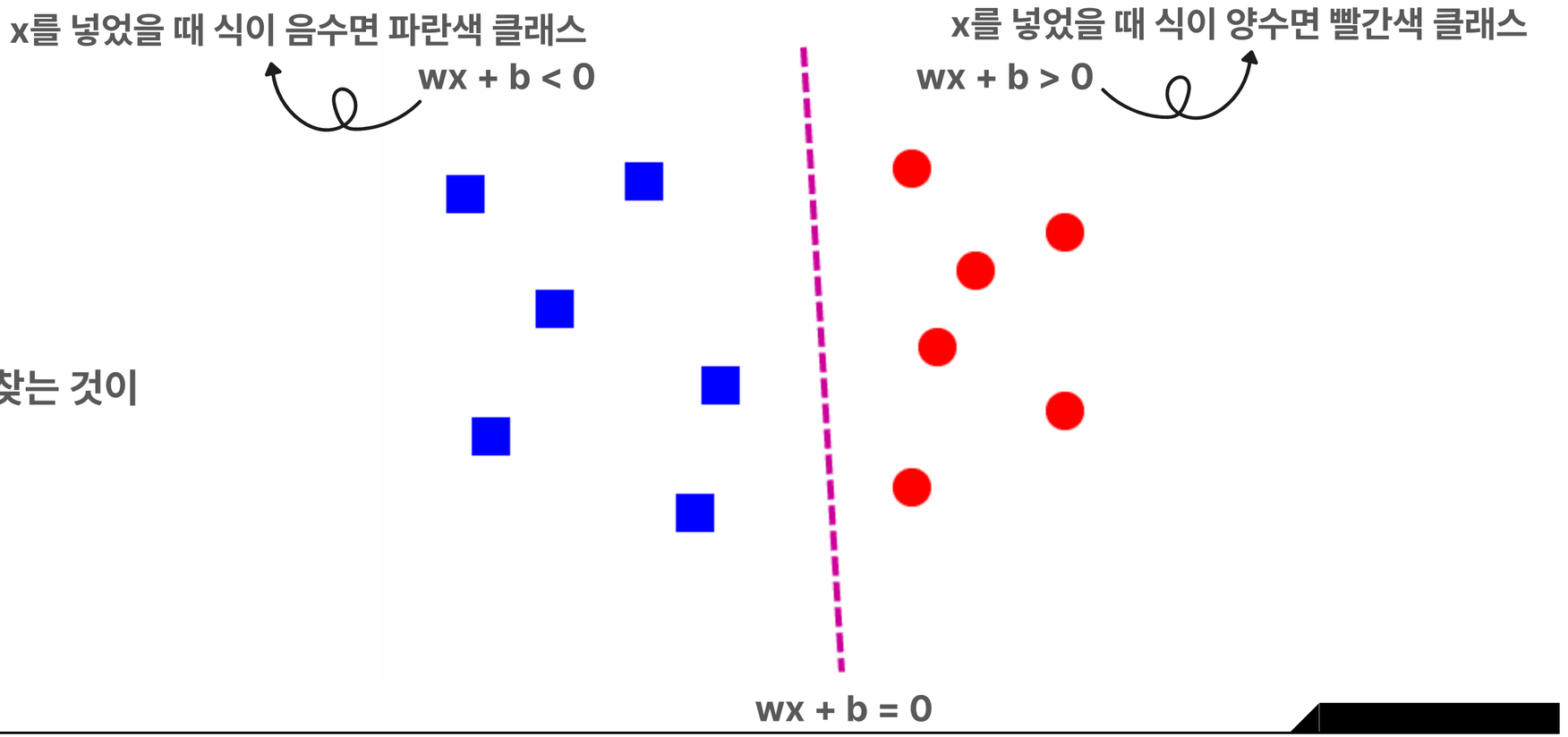
값이 클수록 두 벡터가 유사

점 A(원점에서 출발한 벡터) = [2, 3, 4]
점 B(원점에서 출발한 벡터) = [1, 1, 2]

Cosine Similarity = (2 x 1 + 3 x 1 + 4 x 2) / (벡터 A 크기) x (벡터 B 크기)
= 13 / (root(29) x root(6))

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



Linear Classifier

클래스 간 분류 기준 선(결정 경계선)을 찾는 것이
학습인 분류 모델

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

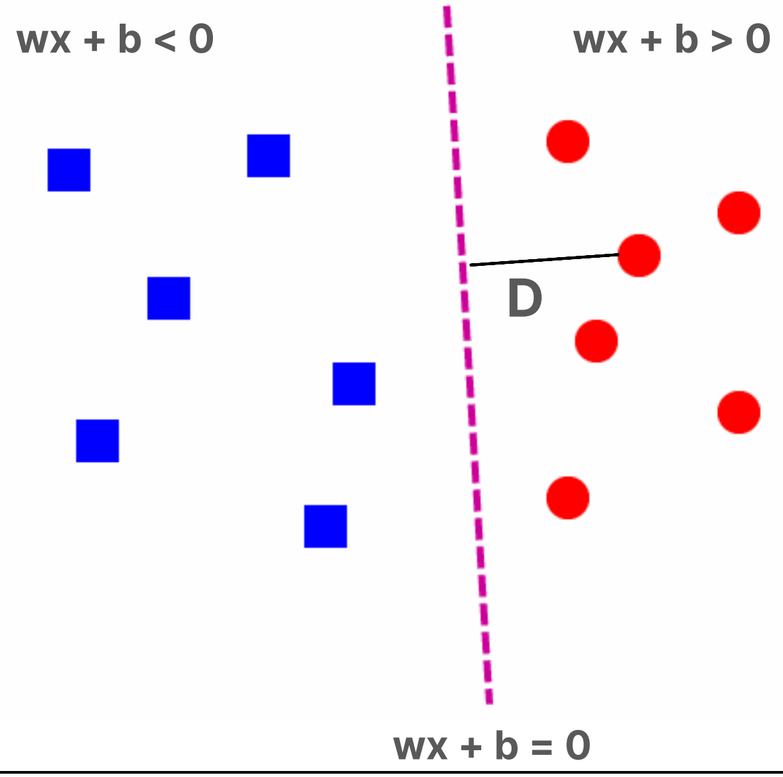
여기서부터 굉장히 복잡한 식이 나옵니다...

다만, 여기서 나오는 식들 대부분은 SVM에서만 사용되는 애들이기에 전체적인 논리 흐름만 따라오시면 됩니다!!

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

D (결정 경계선과의 거리)



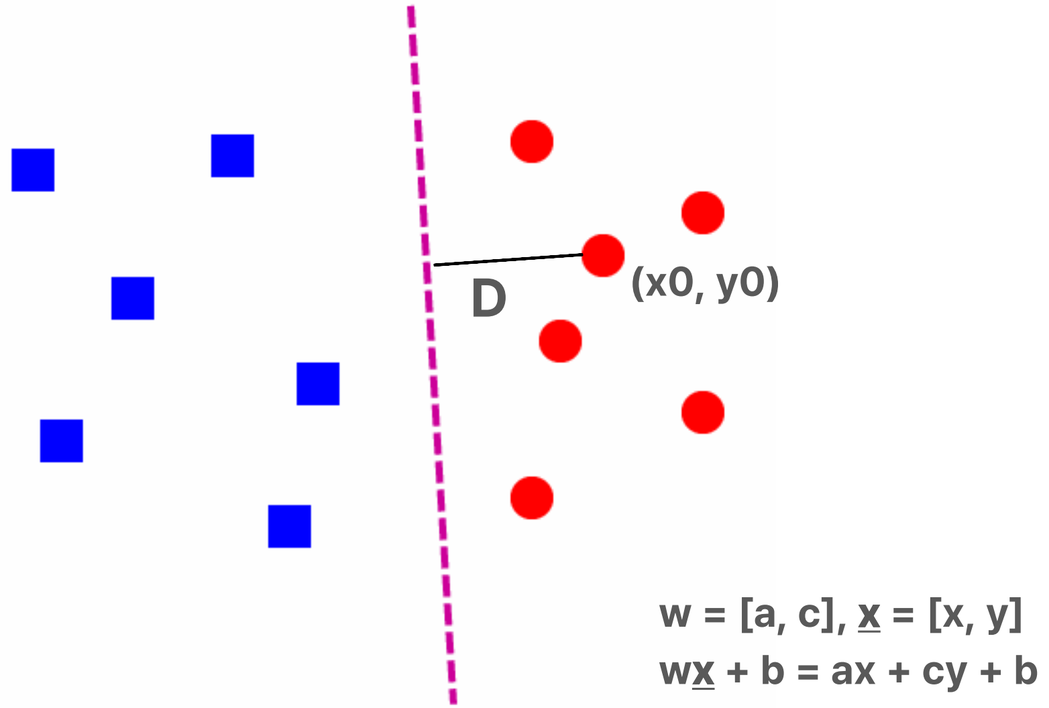
Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

D (결정 경계선과의 거리)

$$D = \frac{|ax_0 + cy_0 + b|}{\sqrt{a^2 + c^2}}$$

고등학교 수학



w = [a, c], x = [x, y]

wx + b = ax + cy + b

wx + b = ax + cy + b = 0

식을 a, b, c로도 표현해봅시다

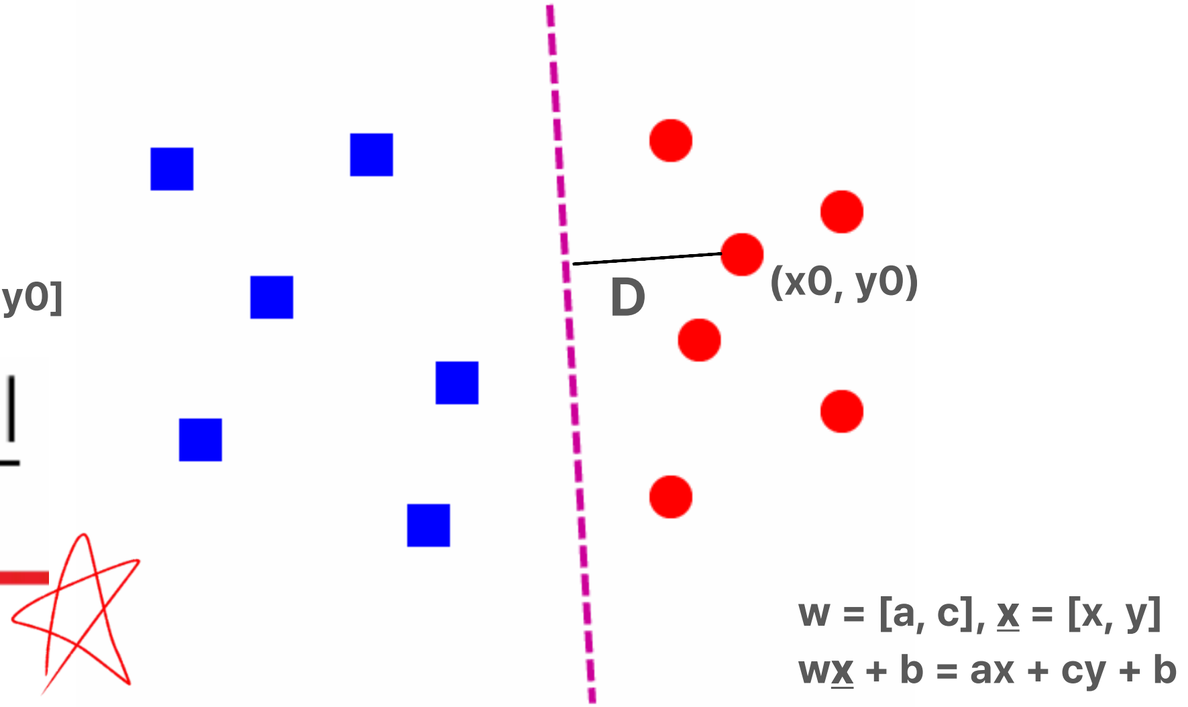
Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

D (결정 경계선과의 거리)

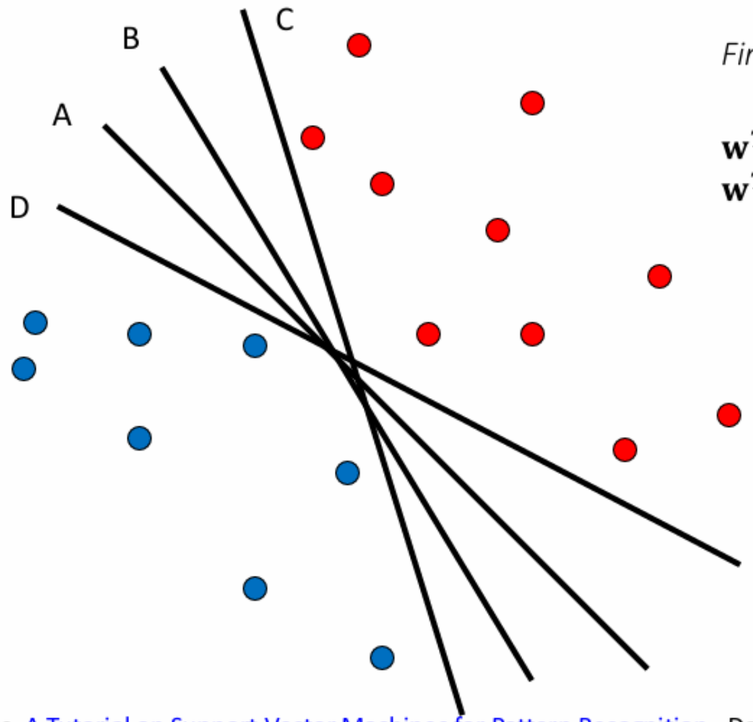
$$D = \frac{|ax_0 + cy_0 + b|}{\sqrt{a^2 + c^2}} = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|}$$

x = [x0, y0]



Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



Find $\mathbf{w} = \begin{bmatrix} a \\ c \end{bmatrix}$ and b such that

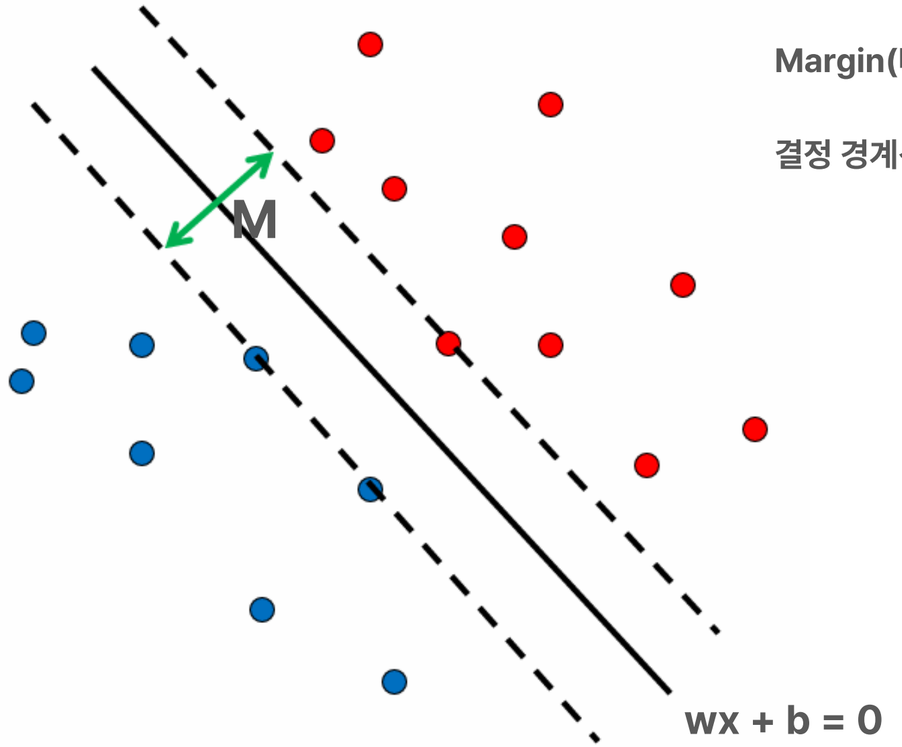
$\mathbf{w}^T\mathbf{x}_i + b > 0 \Rightarrow \mathbf{x}_i$ is positive
$\mathbf{w}^T\mathbf{x}_i + b < 0 \Rightarrow \mathbf{x}_i$ is negative

Which line is best? **목적 함수와 관련있는 질문**

C. Burges, A Tutorial on Support Vector Machines for Pattern Recognition, Data Mining and Knowledge Discovery, 1998

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



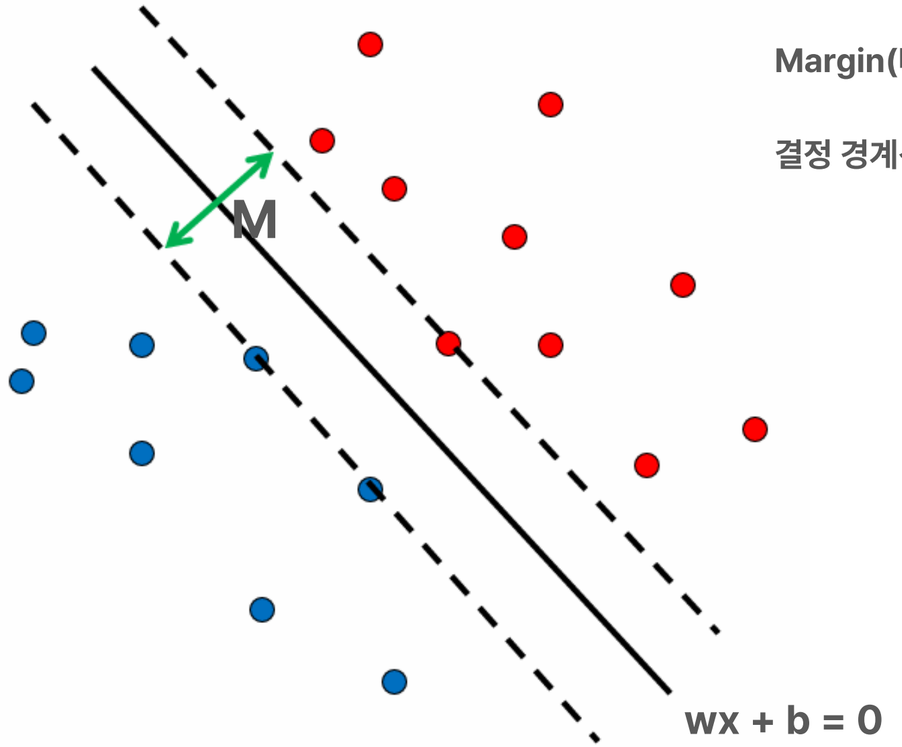
Margin(마진)

결정 경계선과 가장 가까운 positive 샘플 사이 거리 + 결정 경계선과 가장 가까운 negative 샘플 사이 거리

마진을 최대화 시키는 w, b를 찾는 것이 학습의 목적이 됨

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



Margin(마진)

결정 경계선과 가장 가까운 positive 샘플 사이 거리 + 결정 경계선과 가장 가까운 negative 샘플 사이 거리

마진을 최대화 시키는 w, b를 찾는 것이 학습의 목적이 됨

but... 마진 최대화는 좋은데

$wx + b = 0$

$2wx + 2b = 0$

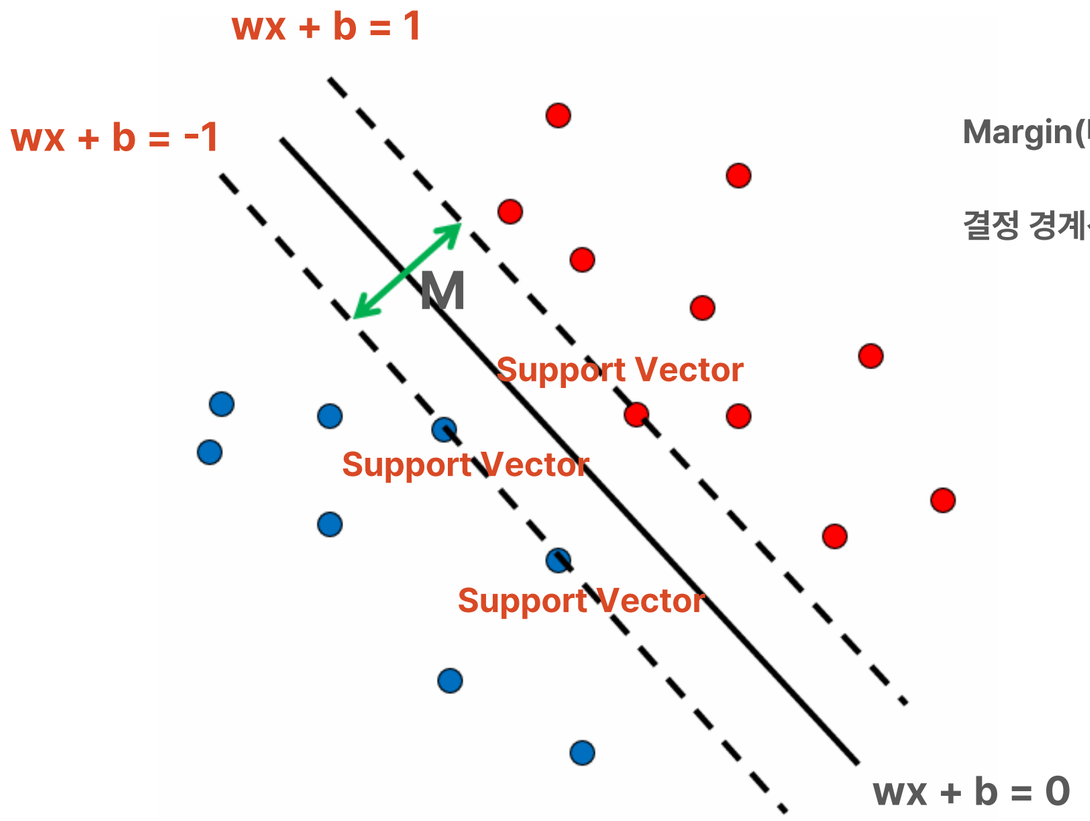
$3wx + 3b = 0$

모두 같은 결정 경계이지만 w와 b 조합이 여러 개가 나올 수 있음

→ 제약 조건을 임의로 하나 더 추가해서 w, b가 하나로 결정될 수 있도록 하자

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



Margin(마진)

결정 경계선과 가장 가까운 positive 샘플 사이 거리 + 결정 경계선과 가장 가까운 negative 샘플 사이 거리

마진을 최대화 시키는 w, b를 찾는 것이 학습의 목적이 됨

제약 조건을 임의로 하나 더 추가해서 w, b가 하나로 결정될 수 있도록 하자

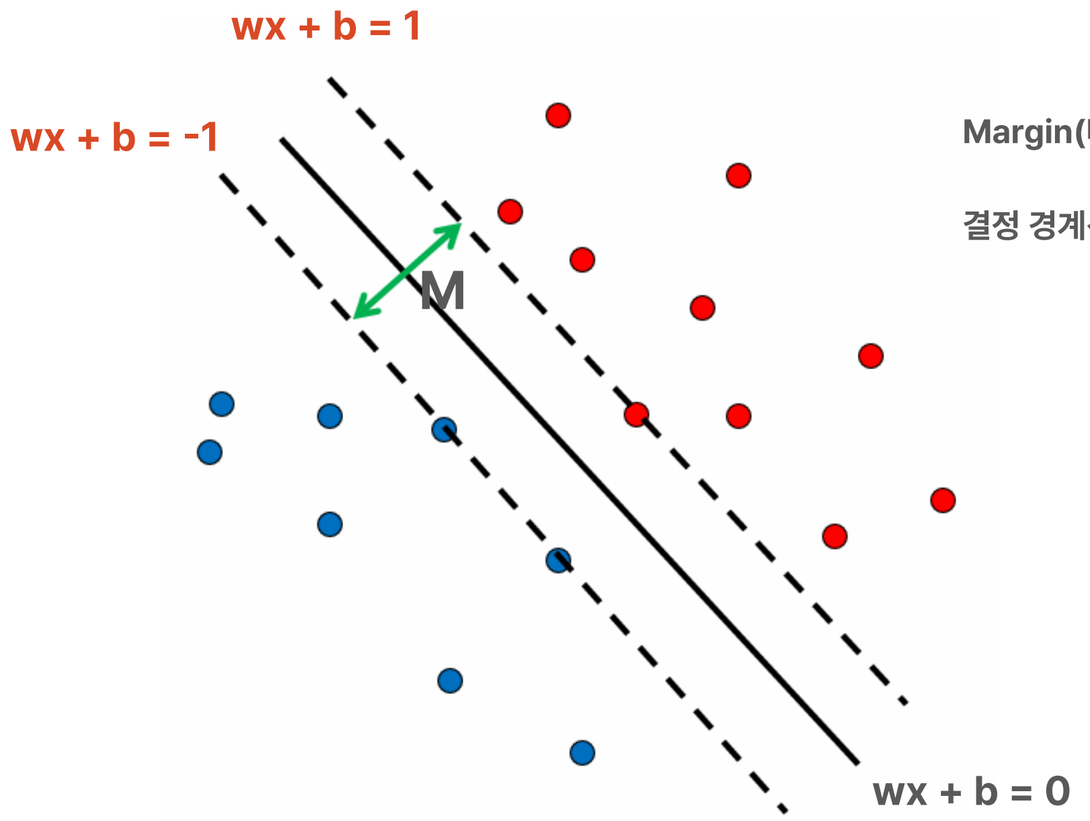
같은 결정 경계를 결정하는 수 많은 w, b 조합 중에

해당 경계선과 가장 가까운 샘플(= Support Vector)의 함숫값이 +1 or -1을

만족하는 경계선으로만 제한 (wx + b = 1 or -1)

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



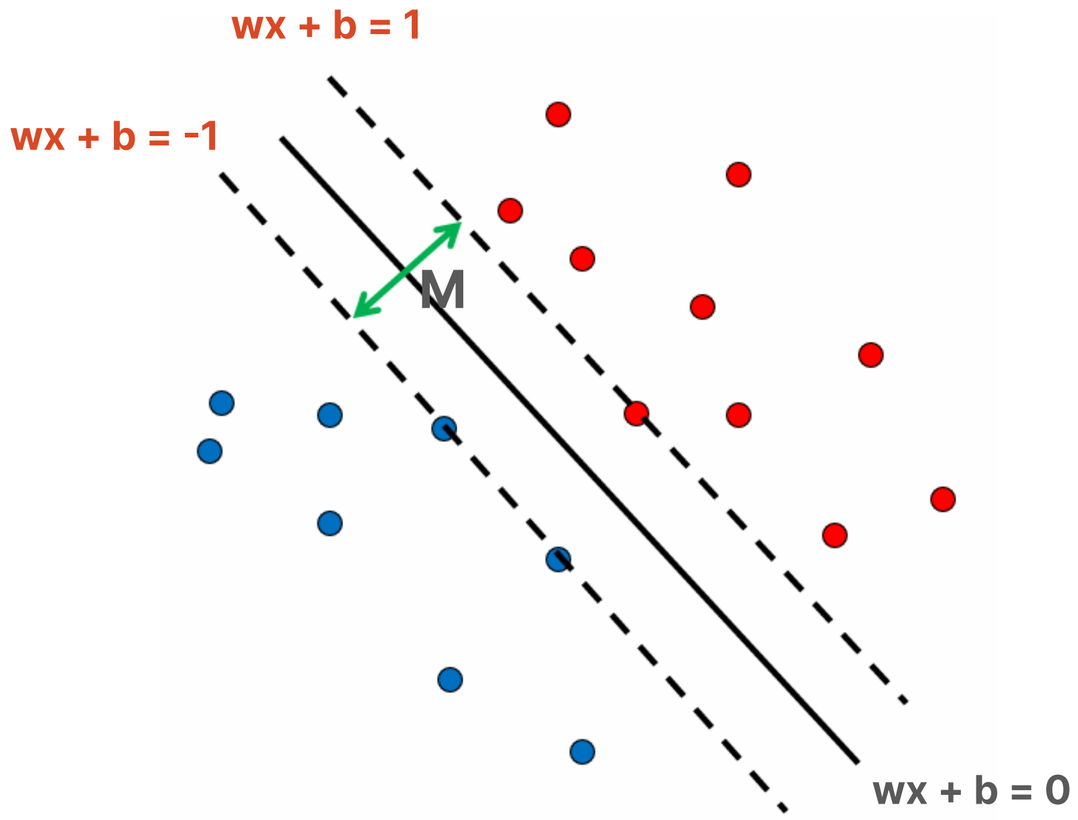
Margin(마진)

결정 경계선과 가장 가까운 positive 샘플 사이 거리 + 결정 경계선과 가장 가까운 negative 샘플 사이 거리

$$M = \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{\|\mathbf{w}\|} + \frac{|\mathbf{w}^T\mathbf{x}_i + b|}{\|\mathbf{w}\|}$$
$$= \frac{1}{\|\mathbf{w}\|} + \frac{1}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}$$

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



SVM: Solve for $\mathbf{w}$ *such that*

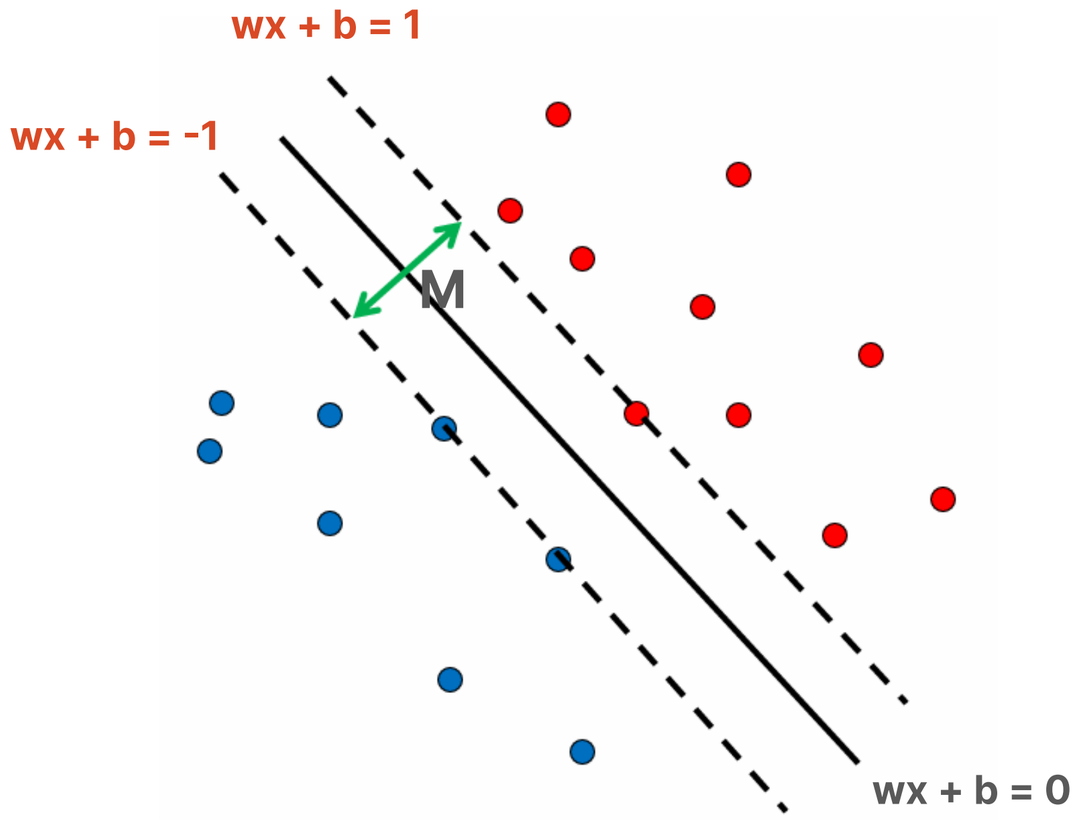
1. $\mathbf{w}^T\mathbf{x}_i + b \geq 1 \Rightarrow \mathbf{x}_i$ is positive $(y_i = +1)$
2. $\mathbf{w}^T\mathbf{x}_i + b \leq -1 \Rightarrow \mathbf{x}_i$ is negative $(y_i = -1)$
3. *The margin* $M = \frac{2}{\|\mathbf{w}\|}$ *is maximized*

따라서, SVM 학습 시 학습 데이터들이 1번 2번을 만족하면서 M을 최대화하는 W와 b를 찾아야 함

* Inference 때는 wx + b가 0보다 크면 positive, 0보다 작으면 negative로 판단 (훈련할 때는 기준을 1로 빡세게 잡고 inference는 기준을 0으로 잡음)

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)



SVM: Solve for $\mathbf{w}$ *such that*

1. $\mathbf{w}^T\mathbf{x}_i + b \geq 1 \Rightarrow \mathbf{x}_i$ is positive $(y_i = +1)$
2. $\mathbf{w}^T\mathbf{x}_i + b \leq -1 \Rightarrow \mathbf{x}_i$ is negative $(y_i = -1)$
3. $\|\mathbf{w}\|$ *is minimized*

따라서, SVM 학습 시 학습 데이터들이 1번 2번을 만족하면서 W의 크기를 최소화해야 함

* Inference 때는 wx + b가 0보다 크면 positive, 0보다 작으면 negative로 판단

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

Further simplification of the condition 1 and 2:

$\mathbf{w}^T\mathbf{x}_i + b \geq +1$ for $\mathbf{x}_i$ positive $(y_i = +1)$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \cdot y_i$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

$\mathbf{w}^T\mathbf{x}_i + b \leq -1$ for $\mathbf{x}_i$ negative $(y_i = -1)$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \leq -1 \cdot y_i = -1 \cdot -1$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

For both $\boldsymbol{x}_i$ positive $(y_i = +1)$ and $\boldsymbol{x}_i$ negative $(y_i = -1)$,
$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

SVM: Solve for $\mathbf{w}$ *such that*

1. $\mathbf{w}^T\mathbf{x}_i + b \geq 1 \Rightarrow \mathbf{x}_i$ is positive $(y_i = +1)$
2. $\mathbf{w}^T\mathbf{x}_i + b \leq -1 \Rightarrow \mathbf{x}_i$ is negative $(y_i = -1)$
3. $\|\mathbf{w}\|$ *is minimized*

따라서, SVM 학습 시 학습 데이터들이 1번 2번을 만족하면서 W의 크기를 최소화해야 함

*** Inference 때는 wx + b가 0보다 크면 positive, 0보다 작으면 negative로 판단**

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

Further simplification of the condition 1 and 2:

$\mathbf{w}^T\mathbf{x}_i + b \geq +1$ for $\mathbf{x}_i$ positive $(y_i = +1)$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \cdot y_i$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

$\mathbf{w}^T\mathbf{x}_i + b \leq -1$ for $\mathbf{x}_i$ negative $(y_i = -1)$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \leq -1 \cdot y_i = -1 \cdot -1$
$\Rightarrow y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

For both $\boldsymbol{x}_i$ positive $(y_i = +1)$ and $\boldsymbol{x}_i$ negative $(y_i = -1)$,
$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

SVM: Solve for $\mathbf{w}$ *such that*

1. $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$
2. $\|\mathbf{w}\|$ *is minimized*

따라서, SVM 학습 시 학습 데이터들이 1번을 만족하면서 W의 크기를 최소화해야 함

*** Inference 때는 wx + b가 0보다 크면 positive, 0보다 작으면 negative로 판단**

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

$$\|\mathbf{w}\| \propto \|\mathbf{w}\|^2 = \mathbf{w}^T\mathbf{w}$$

SVM: Solve for $\mathbf{w}$ *such that*

1. $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$
2. $\|\mathbf{w}\|$ *is minimized*

따라서, SVM 학습 시 학습 데이터들이 1번을 만족하면서 W의 크기를 최소화해야 함

*** Inference 때는 wx + b가 0보다 크면 positive, 0보다 작으면 negative로 판단**

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

$$\|\mathbf{w}\| \propto \|\mathbf{w}\|^2 = \mathbf{w}^T\mathbf{w}$$

1/2는 어차피 목적함수 최적화에 영향을 주지 않기 때문에 임의로 사용

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2$$

$$\text{s.t.} \quad y^{(n)}\left(w^\top x^{(n)} + b\right) \geq 1, \quad n = 1, \ldots, N$$

such that

따라서, SVM 학습 시 학습 데이터들이 부등식을 만족하면서 W의 크기를 최소화하는 W와 b를 찾아야 함.

* Inference 때는 wx + b가 0보다 크면 positive, 0보다 작으면 negative로 판단

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

목적 함수

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2$$
$$\text{s.t.} \quad y^{(n)}\left(w^\top x^{(n)} + b\right) \geq 1, \quad n = 1, \ldots, N$$

부등식 조건이 포함된 목적 함수 최적화 문제의 해 → 라그랑지안 사용

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

목적 함수

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2$$
$$\text{s.t.} \quad y^{(n)}\left(w^\top x^{(n)} + b\right) \geq 1, \quad n = 1, \dots, N$$

부등식 조건이 포함된 목적 함수 최적화 문제의 해 → 라그랑지안 사용

대충 부등식이나 등식 조건이 있는 목적 함수를 조건 없는 새로운 목적 함수로 바꾸는 전략

Lagrange Duality

- The **primal optimization problem** that we want to focus on is

$$\min_{w} \quad f(w)$$
$$\text{s.t.} \quad g_i(w) \leq 0, \quad i = 1, \dots, k$$
$$h_i(w) = 0, \quad i = 1, \dots, l$$

- To solve the problem, **generalized Lagrangian** is defined by

$$\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_{i=1}^{k} \alpha_i g_i(w) + \sum_{i=1}^{l} \beta_i h_i(w)$$

  - where α_i's and β_i's are the Lagrange multipliers

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

새로운 목적 함수(최소화 해야 함)

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{n=1}^{N} \alpha_n \left[y^{(n)} \left(w^\top x^{(n)} + b \right) - 1 \right]$$

α(>=0): Lagrange multipliers

데이터 샘플 x마다 다른 값을 할당함

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

새로운 목적 함수(최소화 해야 함)

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{n=1}^{N} \alpha_n \left[y^{(n)} \left(w^\top x^{(n)} + b \right) - 1 \right]$$

α(>=0): Lagrange multipliers

데이터 샘플 x마다 다른 값 할당

↓

W에 대해 미분 = 0

b에 대해 미분 = 0

$$w = \sum_{n=1}^{N} \alpha_n y^{(n)} x^{(n)} \qquad \sum_{n=1}^{N} \alpha_n y^{(n)} = 0$$

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

새로운 목적 함수(최소화 해야 함)

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{n=1}^{N} \alpha_n \left[y^{(n)} \left(w^\top x^{(n)} + b \right) - 1 \right]$$

α(>=0): Lagrange multipliers

데이터 샘플 x마다 다른 값 할당

대입하면 α에 대한 식으로 바뀜

SMO 알고리즘으로, 목적 함수를 최적화하는

각 데이터 샘플별 α값을 구함

W에 대해 미분 = 0

b에 대해 미분 = 0

$$w = \sum_{n=1}^{N} \alpha_n y^{(n)} x^{(n)} \qquad \sum_{n=1}^{N} \alpha_n y^{(n)} = 0$$

SMO 알고리즘은 설명 생략....

SVM은 전체적인 논리 흐름만 이해하면 좋을 것 같습니다!

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

새로운 목적 함수(최소화 해야 함)

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{n=1}^{N} \alpha_n \left[y^{(n)} \left(w^\top x^{(n)} + b \right) - 1 \right]$$

↓

W에 대해 미분 = 0

b에 대해 미분 = 0

결론! $w = \sum_{n=1}^{N} \alpha_n y^{(n)} x^{(n)}$ $\sum_{n=1}^{N} \alpha_n y^{(n)} = 0$ +

SMO 알고리즘

↓

α > 0: Support Vector

α = 0: Support Vector 외의 데이터 샘플



b 최적값은 w를 구하면 구할 수 있음

$$b = y_i - \mathbf{w}^T \mathbf{x}_i.$$

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)



α > 0: Support Vector

α = 0: Support Vector 외의 데이터 샘플

분류 알고리즘 2: SVM (Support Vector Machine)

SVM Inference 시 모델 내부 계산식

$$f(\mathbf{x}_i) = sign(\mathbf{w}^T\mathbf{x} + b)$$

$$= sign\left(\sum_i \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b\right)$$

sample

Recap

$w = \sum_{n=1}^{N} \alpha_n y^{(n)} x^{(n)}$

sign(x) = 1 (if x > 0) / -1 (if x < 0)

Inference 때, support vector들과 sample 사이 연산 취해서 연산 값이 양수면 1, 음수면 -1로 판단

(support vector가 아닌 학습 데이터들은 알파 값이 0이기 때문에 연산에 영향을 주지 않음)

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

α > 0: Support Vector

α = 0: Support Vector 외의 데이터 샘플

분류 알고리즘 2: SVM (Support Vector Machine)

SVM Inference 시 모델 내부 계산식에 대한 직관적 해석

$$f(\mathbf{x}) = sign(\alpha_1 y_1 \mathbf{x}_1^T \mathbf{x} + \alpha_2 y_2 \mathbf{x}_2^T \mathbf{x} + \alpha_3 y_3 \mathbf{x}_3^T \mathbf{x} + b) = \text{positive}$$

> 0

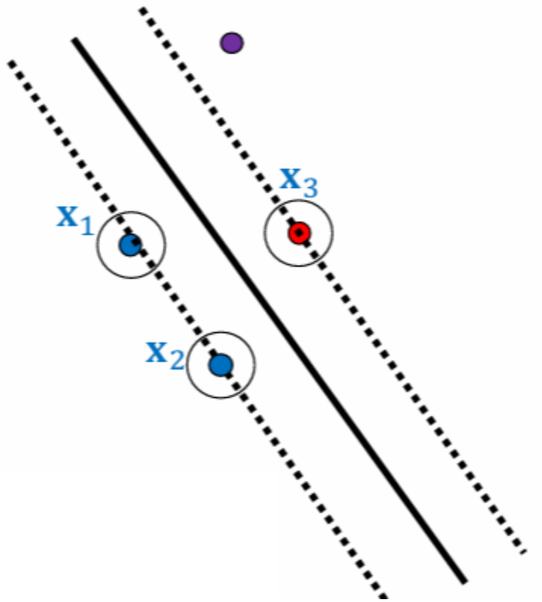
> 0

> 0

벡터의 내적은 벡터 사이 유사도를 내포

Inference 식을 보면 support vector와 샘플 사이 내적 연산을 포함

가까우면 내적 값이 대충 양수, 멀면 내적 값이 대충 음수라고 생각해보면 직관적으로 와 닿음



Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)



α > 0: Support Vector

α = 0: Support Vector 외의 데이터 샘플

분류 알고리즘 2: SVM (Support Vector Machine)

Non-Linear SVM (참고)

결정 경계선이 직선이 아닌 곡선 (좀 더 복잡한 데이터 분포일 때 유효)

x를 다차원 공간으로 매핑하는 함수

$$f(x) = \text{sign}\left(\sum_i \alpha_i y_i \, \phi(x_i)^\top \phi(x) + b\right)$$

연산량 너무 많음

$$K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j)$$

Examples of kernel functions

- Linear: $K(x_i, x_j) = x_i^T x_j$
- Polynomials of degree up to *d*: $K(x_i, x_j) = (x_i^T x_j + 1)^d$
- Gaussian RBF: $K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$

두 내적을 이미 잘 설계가 된 kernel function으로 표현이 가능

내적을 할 필요 없이 kernel function에 원래 feature 인풋만 넣으면 됨

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x): 추가 슬라이드

분류 알고리즘 2: SVM (Support Vector Machine)

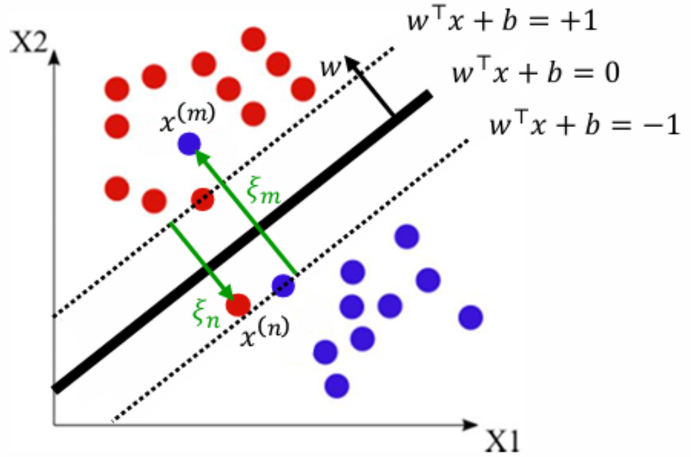
Soft-Margin SVM: 이때까지 본 것은 margin 기준을 엄격하게 잡은 Hard-Margin SVM임

Soft-margin SVM

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2 + C\sum_{n=1}^{N}\xi_n$$

$$\text{s.t.} \quad y^{(n)}\left(w^\top x^{(n)} + b\right) \geq 1 - \xi_n, \quad n = 1, \ldots, N$$

$$\xi_n \geq 0, \quad n = 1, \ldots, N$$



데이터 분포가 굉장히 복잡하면, margin 기준을 엄격히 잡았을 때 분류 경계선을 못 찾고 error가 발생할 수 있음 (training 중에)

그래서 margin 기준을 flexible하게 잡은 SVM: 훈련 데이터가 결정 경계선을 넘어 다른 클래스의 영역에 배치될 수 있음

→ 그럴 경우, penalty가 적용되어 목적 함수가 커짐 (목적 함수의 학습 방향은 최소화)

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 2: SVM (Support Vector Machine)

SVM을 정확하게 이해하기보다는 논리 흐름이 중요 (목적 함수 최적화 관점에서의)

이진 분류에 주로 쓰이지만 SVM을 여러 대 두어 multi-class 분류에 굳이 사용해볼 수는 있음

sklearn에서 바로 불러와 써볼 수 있음



이제는 많이 쓰이지 않지만 가끔 성능 비교 대상 또는 데이터 퀄리티 분석을 위해 간단하게 써보는 등 간간이 사용

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression Inference

x ⇨ 특정 클래스일 확률 예측 ⇨ 확률이 가장 큰 Class로 최종 분류

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression Inference

Logit: 확률 계산되기 직전 모델에서 계산된 값

x ⇨ LinearRegression으로 logit 계산 ⇨ 특정 클래스일 확률 예측 ⇨ Class 분류



머신러닝이나 딥러닝에서 확률 값을 예측하려면 어떻게 해야할까?

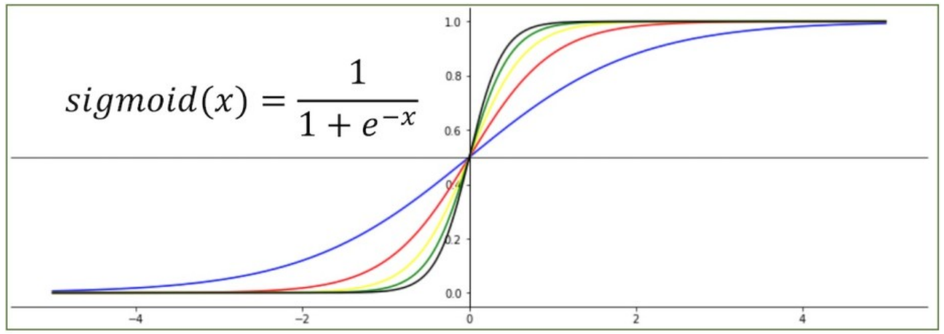
Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)



머신러닝이나 딥러닝에서 확률 값을 예측하려면 어떻게 해야할까?

이진 분류: Sigmoid 함수 사용



x = 모델에서 확률 예측 전까지 계산된 값 (logit)

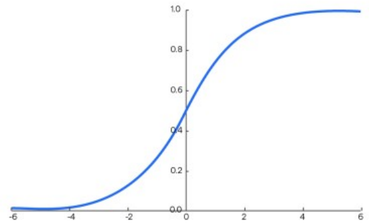
$x \in R^{1\times1}$

Sigmoid를 통과해도 차원은 그대로: 확률 값이 하나만 출력됨
남은 class는 1에서 해당 확률을 빼면 구해질 수 있음

이 두 함수의 값은 인풋 logit에 비례하고
0과 1사이의 값을 반환하기 때문에
확률값 예측에 적절

Multi-class 분류: Softmax 함수 사용

Softmax Function



$$y_k = \frac{e^{x_k}}{\sum_{i=1}^{n} e^{x_i}}$$

x = 모델에서 확률 예측 전까지 계산된 값 (logit)

$x \in R^{\text{class 수 x 1}}$

Softmax를 통과해도 차원은 그대로: 확률 값이 클래스 수만큼 벡터로 출력

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)



머신러닝이나 딥러닝에서 확률 값을 예측하려면 어떻게 해야할까?

이진 분류: Sigmoid 함수 사용

Multi-class 분류: Softmax 함수 사용

모델에서 확률 예측 전까지 계산된 값 (logit)

x → 이진 분류: Sigmoid 함수 / Multi-class 분류: Softmax 함수 → Class별 확률 값 반환

그렇다면 Logistic Regression의 **목적 함수**는?

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression의 **목적 함수:** 두 가지의 다른 경로로 동일한 형태의 식을 얻을 수 있음

1. Likelihood

2. KL Divergence

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression의 목적 함수: from Likelihood

Likelihood란

학습이 최대화 시켜야 하는 확률
(학습이 어떤 확률을 최대화 시켜야 우리가 하고자 하는 task를 완벽히 수행할까 목적 함수 설계 시 고민이 필요)

Likelihood를 수식으로 정의하고 그것을 목적 함수로 하여 이를 최대화하는 파라미터를 찾게 하는 것
(목적 함수 설계 방법 중의 하나)

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression의 목적 함수: from Likelihood

여기서 Likelihood는 어떻게 정의 내릴 수 있을까?

$$P(\mathcal{D}) = \prod_{i=1}^{N} \prod_{k=1}^{K} \left(p_k^{(i)} \right)^{y_k^{(i)}}$$

yk: class k가 정답인지 아닌지 (정답이면 1, 오답이면 0)

pk: class k일 확률

i: i번째 데이터 샘플

→ 각 샘플 별로 모델이 예측한 정답 클래스일 확률을 곱의 법칙으로 연결한 형태
→ 이걸 최대화 시켜야 모델이 정답 클래스를 더 예측을 잘할 것임

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression의 목적 함수: from Likelihood

여기서 Likelihood는 어떻게 정의 내릴 수 있을까?

$$P(\mathcal{D}) = \prod_{i=1}^{N} \prod_{k=1}^{K} \left(p_k^{(i)}\right)^{y_k^{(i)}}$$

곱셈은 미분하기 불편 → 덧셈으로 바꿈 (log)

$$\log P(\mathcal{D}) = \sum_{i=1}^{N} \sum_{k=1}^{K} y_k^{(i)} \log p_k^{(i)}$$

최대화 문제를 loss의 개념처럼 최소화 문제로 바꾸고 싶음 → 마이너스 붙임

최종 목적 함수(Likelihood → Loss)

Cross-Entropy

$$\mathcal{L} = -\sum_{i=1}^{N} \sum_{k=1}^{K} y_k^{(i)} \log p_k^{(i)}$$

→ 각 샘플 별로 모델이 예측한 정답 클래스일 확률을 곱의 법칙으로 연결한 형태
→ 이걸 최대화 시켜야 모델이 정답 클래스를 더 예측을 잘할 것임

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression의 목적 함수: from KL divergence

KL Divergence

두 확률 분포 사이의 difference를 나타냄
(값이 작을 수록 유사함, 다만 거리 개념은 x)

KL Divergence $\mathrm{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$

class 2일 확률은 1

class 0일 확률은 0

p(x): Human label distribution (Ex. [0, 0, 1, 0, 0])

Q(x): Model Prediction distribution (Ex. [0.2, 0.1, 0.5, 0.1, 0.1])

이 식을 최소화는 것 = KL divergence를 최소화 = Model prediction 분포를 정답 분포에 근접 시킴

$$\mathrm{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \Rightarrow -\sum_x p(x) \log p(x) + \sum_x p(x) \log \frac{p(x)}{q(x)} \Rightarrow -\sum_x p(x) \log q(x) \Rightarrow \mathcal{L} = -\sum_{i=1}^{N} \sum_{k=1}^{K} y_k^{(i)} \log p_k^{(i)}$$

이 term은 사실 Entropy 식임....!

p(x)와 관련된 term을 더해도, 파라미터와
관련 없기 때문에 목적 함수를 바꾸지는 않음

p(x) term이 상쇄

최종 목적 함수

Cross-Entropy

모든 데이터 샘플에 대해 loss sum

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression의 **목적 함수 = Cross-Entropy**

$$\mathcal{L} = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_k^{(i)} \log p_k^{(i)}$$

	Train data sample 1	Train data sample 2	....
정답 확률 분포	[0, 0, 1, 0, 0]	[1, 0, 0, 0, 0]	
모델 예측 확률 분포	[0.2, 0.1, 0.5, 0.1, 0.1]	[0.4, 0.3, 0.1, 0.1, 0.1]	
L	-log(0.5)	-log(0.4)	Total Loss = -log(0.5) - log(0.4) - .....

Classification(분류) Task로 공부하는 머신러닝 (딥러닝 x)

분류 알고리즘 3: Logistic Regression (회귀가 아닌 분류 모델임!!)

Logistic Regression의 **목적 함수 = Cross-Entropy**

$$\mathcal{L} = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_k^{(i)} \log p_k^{(i)}$$

Closed-form이 아니라서 미분 = 0 방식으로 파라미터를 바로 못 구하기에 점진적 학습 필요

경사 하강법을 사용 → 다음 주차 딥러닝 스터디 때 설명 예정

어떻게 보면, 로지스틱 회귀는 딥러닝의 가장 단순한 구성 요소와 구조적으로 비슷함

Regression & Classification을 위한 머신러닝 모델 정리

공부용으로는 알아야 하지만,
실질적으로 많이 사용되는 모델들은 아님

그렇다면 머신러닝 알고리즘 중에서는
쓸만한 게 없는 걸까?

No

Regression

Linear Regression: 회귀 task의 기본 개념 학습용으로 공부

Polynomial Regression: 단순 직선으로 데이터 분포를 학습하기에는 무리라는 것을 확인

Multiple Linear Regression: 예측에는 feature 1개가 아닌 여러 개를 쓰는 경우가 많음

Classification

KNN: 가장 기본적인 분류기, 분류 task의 기본 개념 학습용으로 공부, 거리 metrics

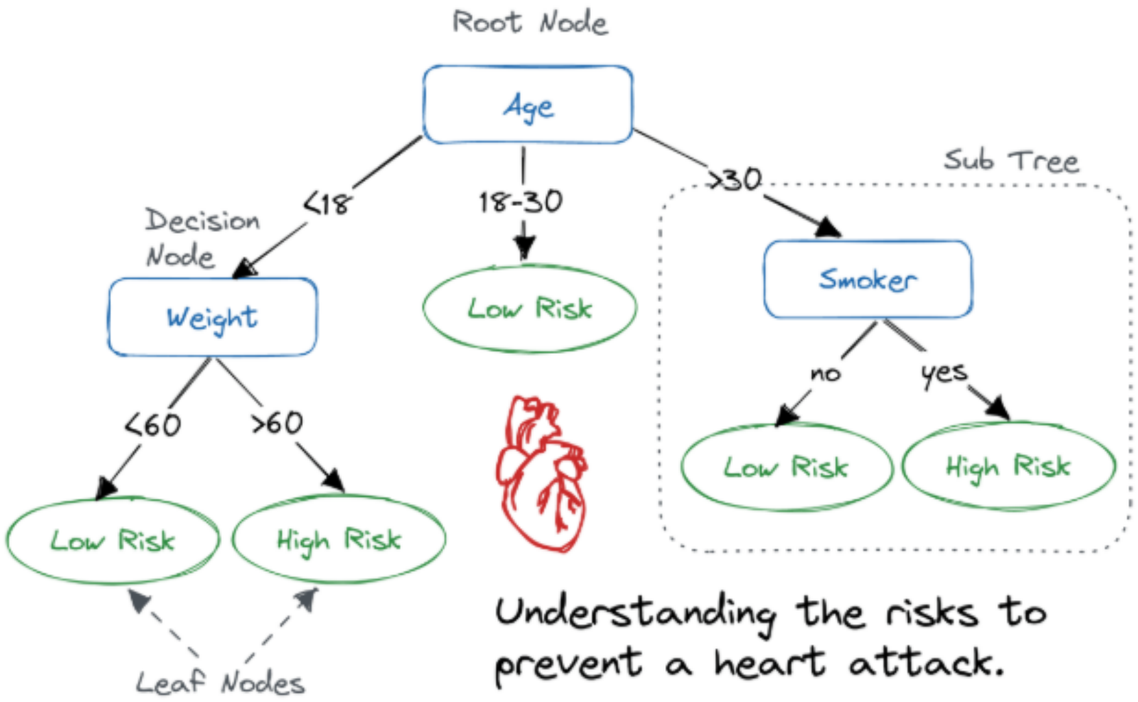
SVM: Closed-form 목적 함수 최적화 흐름 공부

Logistic Regression: 딥러닝을 구성하는 기본 단위 공부, Cross-Entropy 개념 학습용

Decision Tree Algorithm

머신러닝 알고리즘 SOTA 모델들이 이 알고리즘을 사용 (정형 데이터라면 **딥러닝보다도 높은 성능 가능**)

결정 트리 (분류)



Regression & Classification 모두 적용 가능

각 노드에 데이터 샘플을 분할하는 규칙이 명시되어 있음

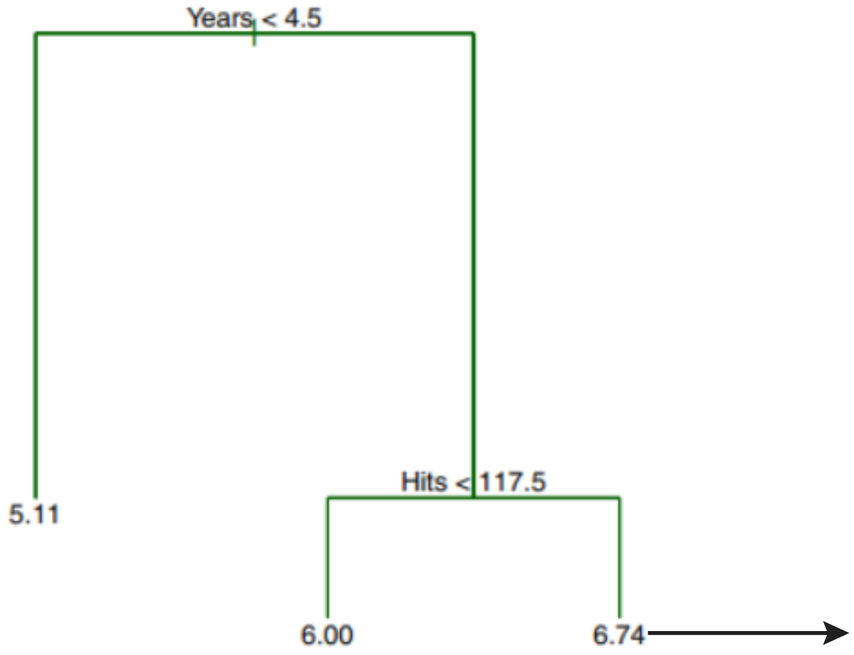
그리고 그 규칙은 입력 feature 1개를 설정해서 만듦 (ex. Age feature < 18)

Inference할 때, 각 데이터 샘플은 이 규칙을 통해
적절한 노드로 이동하면서 최종 결과에 도달함

Decision Tree Algorithm

머신러닝 알고리즘 SOTA 모델들이 이 알고리즘을 사용 (정형 데이터라면 **딥러닝보다도 높은 성능 가능**)

결정 트리 (회귀)



여기에 속한 학습 데이터 정답(y) 값들의 평균: Inference 때 이 노드에 도달한 샘플은 6.74로 예측

Regression & Classification 모두 적용 가능

각 노드에 데이터 샘플을 분할하는 규칙이 명시되어 있음

그리고 그 규칙은 입력 feature 1개를 설정해서 만듦 (ex. Years feature < 18)

Inference할 때, 각 데이터 샘플은 이 규칙을 통해
적절한 노드로 이동하면서 최종 결과에 도달함

Decision Tree Algorithm

머신러닝 알고리즘 SOTA 모델들이 이 알고리즘을 사용 (정형 데이터라면 딥러닝보다도 높은 성능 가능)

결정 트리의 학습

가능한 규칙(feature 한 개에 대한)들을 탐색해서, 해당 규칙을 적용했을 때 train data에서 가장 이상적인 결과가 나오는 규칙을 선택

이상적인 결과?

규칙을 적용해서 샘플들을 분할 했을 때, 분할된 노드에 포함된 샘플들은
한 클래스만 있는 경우가 섞여 있는 경우보다 더 이상적 (분류를 잘했다는 뜻이니깐)

이걸 수치화한 것이 불확실성 지표 (엔트로피 / 지니계수)

엔트로피

x = x번 째 클래스
p(x) = 해당 클래스가 해당 노드에서 차지하는 비율

지니계수

c = 클래스 개수
pi = 클래스 i가 해당 노드에서 차지하는 비율

특히, 엔트로피라는 개념은 굉장히 많이 나옴
엔트로피가 높을수록 모델이 답을
확신하지 못하는 상태라고 생각하면 됨

Logistic regression을 예로 들면... 모델 예측 확률 분포가
[0.2, 0.2, 0.2, 0.2, 0.2]면 엔트로피가 높음
[0.1, 0.1, 0.1, 0.6, 0.1]이면 엔트로피가 낮음

엔트로피

$$-\sum_{x} p(x) \log p(x)$$

지니계수

$$G(S) = 1 - \sum_{i=1}^{c} p_i^2$$

지니 계수가 높을수록 모델이 답을 확신하지 못하는 상태

Decision Tree Algorithm

머신러닝 알고리즘 SOTA 모델들이 이 알고리즘을 사용 (정형 데이터라면 딥러닝보다도 높은 성능 가능)

결정 트리의 학습

가능한 규칙(feature 한 개에 대한)들을 탐색해서, 해당 규칙을 적용했을 때 train data에서 가장 이상적인 결과가 나오는 규칙을 선택

이상적인 결과?

규칙을 적용해서 샘플들을 분할 했을 때, 분할된 노드에 포함된 샘플들은
한 클래스만 있는 경우가 섞여 있는 경우보다 더 이상적 (분류를 잘했다는 뜻이니깐)

이걸 수치화한 것이 불확실성 지표 (엔트로피 / 지니계수)

엔트로피 또는 지니계수가 높다?

모델이 답을 확신하지 못함
= 노드에서 샘플들의 클래스가 많이 섞여 있음

→

엔트로피 또는 지니계수가
최소화되는 규칙을 찾아야 함

Regression task이면, 샘플 분산으로
엔트로피 결정

엔트로피

$$-\sum_{x} p(x) \log p(x)$$

지니계수

$$G(S) = 1 - \sum_{i=1}^{c} p_i^2$$

Decision Tree Algorithm

머신러닝 알고리즘 SOTA 모델들이 이 알고리즘을 사용 (정형 데이터라면 **딥러닝보다도 높은 성능 가능**)

결정 트리: Feature Importance를 제공

계산하는 식들이 중요한 것은 아님! 결정트리 알고리즘은 feature importance를 제공해준다가 중요

정보 이득(Information Gain): 대충 분할 전 엔트로피 - 분할 후 엔트로피

$$IG(D, A) = I(D) - \sum \left(\frac{|D_j|}{|D|} \times I(D_j) \right)$$

여기서:

- $IG(D, A)$는 데이터 집합 D에 대한 속성 A의 정보 이득입니다.
- $I(D)$는 원래 데이터 집합의 불순도입니다.
- $\frac{|D_j|}{|D|}$는 분할된 데이터 집합의 크기 비율입니다.
- $I(D_j)$는 분할된 데이터 집합의 불순도입니다.

next

각 Feature가 예측에 기여하는 정도(Feature Importance)

$$f_i = \frac{\sum_n \mathbf{1}\{x_i\text{가 노드 } n\text{에서 사용됨}\} \cdot \text{gain}_n}{\sum_n \text{gain}_n}$$

n: 노드 번호

1{xi가 노드 n에서 사용됨}: i번째 feature가 노드 n의 규칙에 적용되면 값이 1

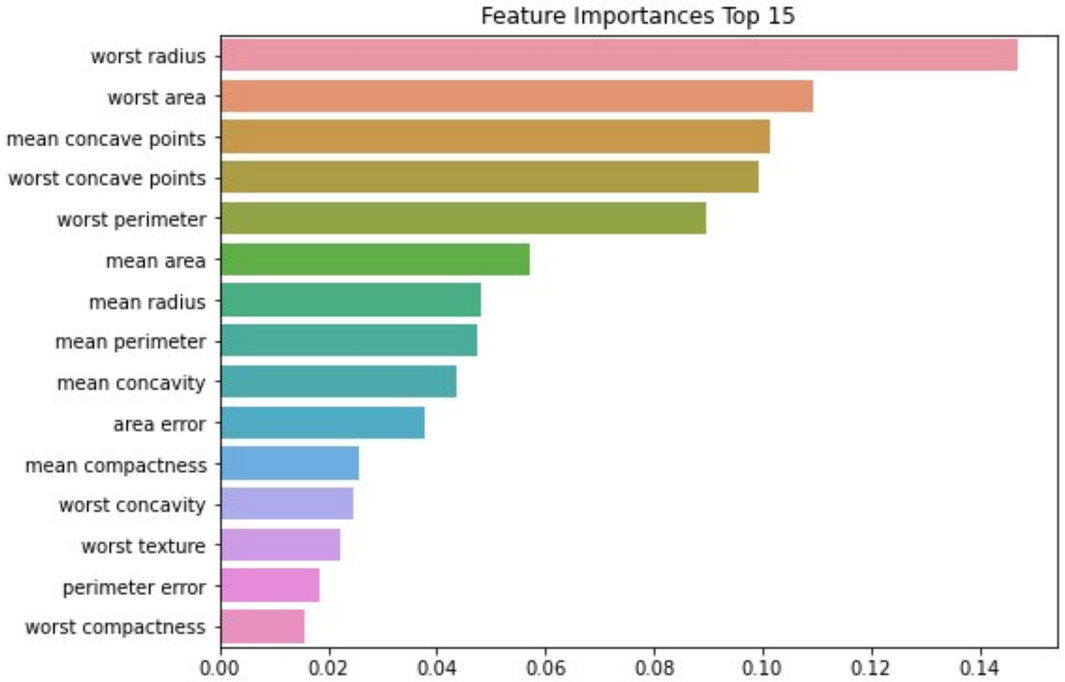
gain_n: n번째 노드에서 정보 이득

분모: 전체 노드 정보 이득 합

Decision Tree Algorithm

머신러닝 알고리즘 SOTA 모델들이 이 알고리즘을 사용 (정형 데이터라면 **딥러닝보다도 높은 성능 가능**)

결정 트리: Feature Importance를 제공



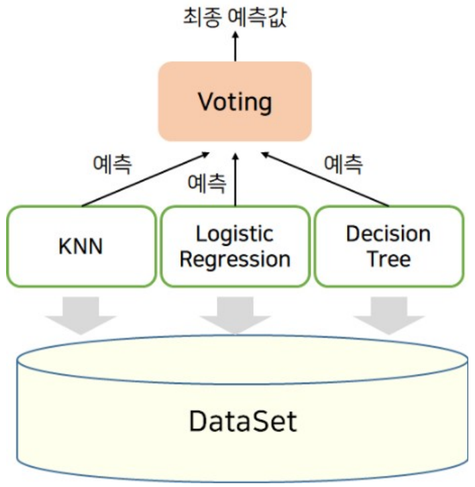
이런 식으로 결정 트리를 학습 시키면,
feature importance를 시각화할 수 있음

Decision Tree Algorithm: Ensemble(앙상블) 기법

앙상블 기법?

Task 수행에 학습된 모델 하나만을 사용하는 것이 아니라, 학습된 모델을 여러 개 두어 결과를 종합해 최종 결과를 도출하는 기법

앙상블 기법 1: Voting



각각 다른 알고리즘이 적용된 모델을 병렬 inference → Voting

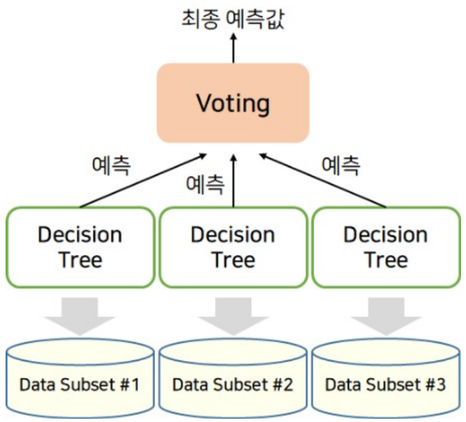
Voting의 예
1. 결과 값 중 가장 개수가 많은 결과 값 선택
2. 모델이 예측한 클래스별 확률 분포를 평균낸 후 가장 높은 확률을 가진 클래스를 최종 클래스로
3. 자유롭게 정의....

Decision Tree Algorithm: Ensemble(앙상블) 기법

앙상블 기법?

Task 수행에 학습된 모델 하나만을 사용하는 것이 아니라, 학습된 모델을 여러 개 두어 결과를 종합해 최종 결과를 도출하는 기법

앙상블 기법 2: Bagging



같은 알고리즘이 적용된 모델(동일 데이터셋 학습 x, 분할 학습 o)을 병렬 inference → Voting

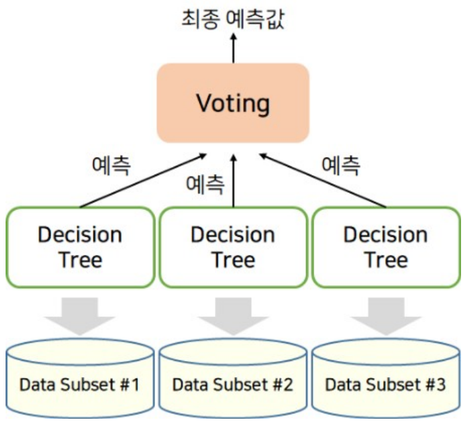
Voting의 예
1. 결과 값 중 가장 개수가 많은 결과 값 선택
2. 모델이 예측한 클래스별 확률 분포를 평균낸 후 가장 높은 확률을 가진 클래스를 최종 클래스로
3. 자유롭게 정의....

Decision Tree Algorithm: Ensemble(앙상블) 기법

앙상블 기법?

Task 수행에 학습된 모델 하나만을 사용하는 것이 아니라, 학습된 모델을 여러 개 두어 결과를 종합해 최종 결과를 도출하는 기법

앙상블 기법 2: Bagging



대표 모델: RandomForest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import RandomForestRegressor
```

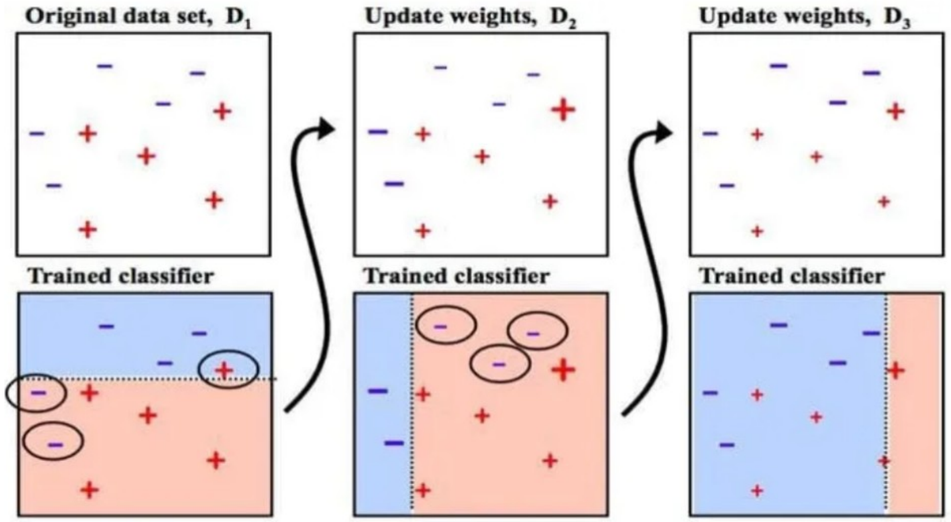
Decision Tree Algorithm: Ensemble(앙상블) 기법

앙상블 기법?

Task 수행에 학습된 모델 하나만을 사용하는 것이 아니라, 학습된 모델을 여러 개 두어 결과를 종합해 최종 결과를 도출하는 기법

앙상블 기법 3: Boosting

정형 데이터 머신러닝 SOTA 모델 = **Decision Tree Algorithm + Boosting**
딥러닝보다 사용도 쉽고(패키지 import 딸깍), 정형 데이터 한정 딥러닝과 성능이 비슷하거나 높을 수도 있음



AdaBoost 모델 (SOTA는 아님, Boosting 기법(모델 순차 학습)의 시작)

1st: 데이터셋을 결정 트리로 학습하고 동일한 데이터셋을 inference 해봄

2nd: 데이터셋 내 오류난 샘플들에 가중치를 부여 (데이터에 가중치를 부여한 것)

3rd: 가중치가 부여된 데이터셋을 다음 모델이 학습 (가중치가 크게 부여된 샘플을 더 유의해서 학습)

4th: 1~3 반복 → 오류가 많이 났던 모델들은 나중에 inference voting을 할 때 결과를 덜 반영

5th: Inference할 때, 각 모델 voting (각 모델 별로 반영되는 비율이 다름)

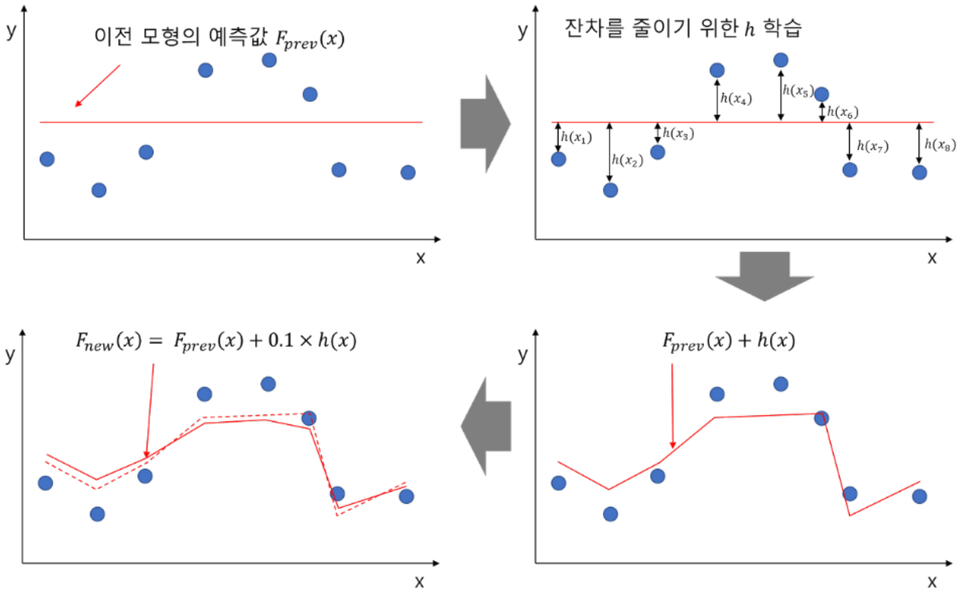
Decision Tree Algorithm: Ensemble(앙상블) 기법

앙상블 기법?

Task 수행에 학습된 모델 하나만을 사용하는 것이 아니라, 학습된 모델을 여러 개 두어 결과를 종합해 최종 결과를 도출하는 기법

앙상블 기법 3: Boosting

정형 데이터 머신러닝 SOTA 모델 = **Decision Tree Algorithm + Boosting**
딥러닝보다 사용도 쉽고(패키지 import 딸깍), 정형 데이터 한정 딥러닝과 성능이 비슷하거나 높을 수도 있음



GBM(Gradient Boosting) 모델 (**SOTA**)

1st: 처음 모델(Fprev)이 학습 데이터에 대해 일단 예측

2nd: 그럼 각 학습 데이터 샘플 별로 잔차(실제 값 - 예측 값)가 생길 것임

3rd: 학습 데이터의 y(정답지)를 잔차로 교체 → 예측 값을 잔차로 하는 새로운 모델(h(x))을 학습

4th: 업데이트 된 모델(Fnew)는 Fprev + (학습률) x h로 정의 (잔차를 성급하게 다 더해주기 보다는 적절히)

5th: 1~4 반복

6th: 최종 모델 = Fprev + (학습률) x h1 + (학습률) x h2 + ...

(== inference할 때, Fprev의 예측 값과 잔차 모델들의 예측 값을 합함)

Decision Tree Algorithm: Ensemble(앙상블) 기법

GBM(Gradient Boosting) 모델 (머신러닝 SOTA)

XGBoost

[Submitted on 9 Mar 2016 (v1), last revised 10 Jun 2016 (this version, v3)]

XGBoost: A Scalable Tree Boosting System

Tianqi Chen, Carlos Guestrin

LightGBM

Microsoft | Research

CatBoost

[Submitted on 28 Jun 2017 (v1), last revised 20 Jan 2019 (this version, v5)]

CatBoost: unbiased boosting with categorical features

패키지 import해서 매우 간단하게 사용 가능 (sklearn에서도 지원)

```
import catboost
import xgboost as xgb
import lightgbm as lgb
```

Decision Tree Algorithm: Ensemble(앙상블) 기법

GBM(Gradient Boosting) 모델 (머신러닝 SOTA)

제 경험...

제가 동아리 친구들과 해커톤 대회를 주최한 적이 있는데
정형 데이터 task 관련해서는 2개의 주제를 개최하였고,
두 task에 대해 순위권에 든 **모든 팀이 LightGBM을 사용**
(모델이 다 같았기에, 결국 데이터 전처리 방식의 싸움)

XGBoost

[Submitted on 9 Mar 2016 (v1), last revised 10 Jun 2016 (this version, v3)]

XGBoost: A Scalable Tree Boosting System

Tianqi Chen, Carlos Guestrin

LightGBM

Microsoft | Research

CatBoost

[Submitted on 28 Jun 2017 (v1), last revised 20 Jan 2019 (this version, v5)]

CatBoost: unbiased boosting with categorical features

패키지 import해서 매우 간단하게 사용 가능 (sklearn에서도 지원)

```
import catboost
import xgboost as xgb
import lightgbm as lgb
```

Hyperparameter vs Parameter

Hyperparameter

모델 내부 변수가 아닌 사용자가 직접 모델 동작 전 넣어주는 상수 값

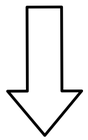
Parameter

사용자가 설정하지 못하는 모델 내부 변수

Hyperparameter vs Parameter

Hyperparameter

모델 내부에서 학습되는 변수가 아닌 사용자가 직접 모델 동작 전 넣어주는 상수 값



모델을 사용하는 사용자가 hyperparameter에
어떤 값을 넣느냐에 따라서도 결과가 확 달라질 수도 있음

최적의 하이퍼파라미터 탐색이 필요

Ex. LightGBM의 하이퍼파라미터

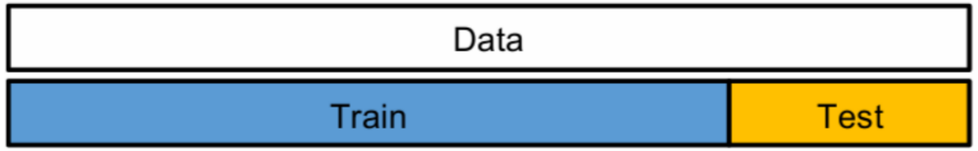
https://lightgbm.readthedocs.io/en/latest/index.html

LightGBM 공식 홈페이지

num_leaves: 최대 말단 노드 수
max_depth: 트리의 최대 깊이
min_data_in_leaf: 말단 노드가 되기 위한 최소 샘플 수
.....

Dataset Split

Case 1) Dataset = Train data + Test data



전체 데이터셋을 train data와 test data로 split

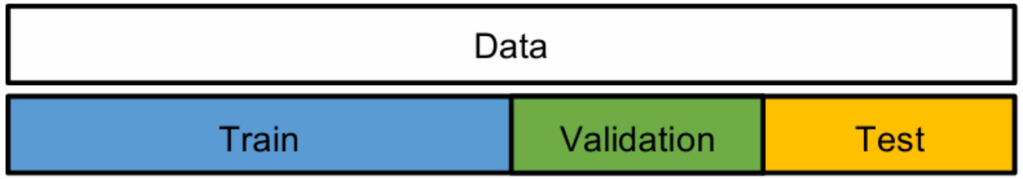
Train data: train할 때 사용할 데이터셋

Test data: 학습 마친 모델을 평가할 때 사용할 데이터셋

```
from sklearn.model_selection import train_test_split
```

Dataset Split

Case 2) Dataset = Train data + Validation data + Test data



전체 데이터셋을 train data와 validation data, test data로 split

Train data: train할 때 사용할 데이터셋

Validation data: 학습 중간 모델 성능 검증용 → 학습 추이 파악(계속 파라미터 업데이트 해도 성능이 안 오른다고 판단되면 학습 조기 종료도 가능) + 최적 하이퍼파라미터 검증

Test data: 학습 마친 모델을 평가할 때 사용할 데이터셋

```
from sklearn.model_selection import train_test_split
```

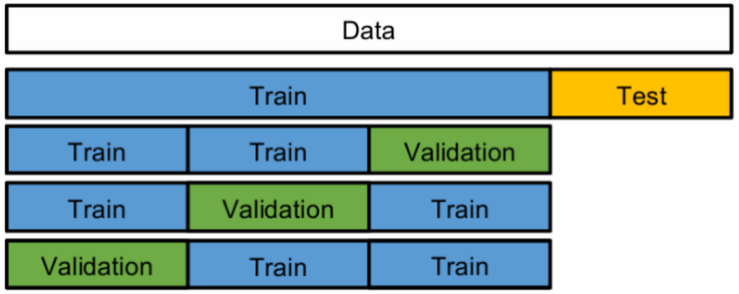
Dataset Split

```
from sklearn.model_selection import KFold
```

or 좀 더 쉽게

```
from sklearn.model_selection import cross_val_score
```

Case 3) K-fold Cross Validation



일단 Train, Test로 split

학습 진행하면서 Train dataset을 K분할 (그림은 3분할): K-1개의 Train data, 1개의 validation data

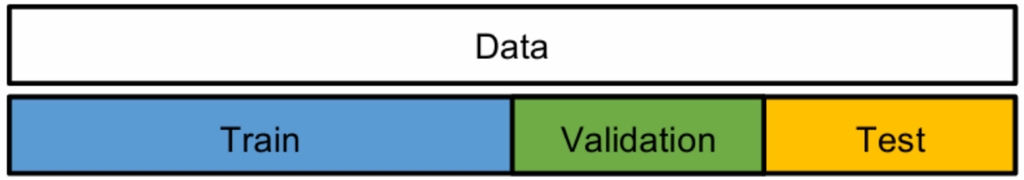
Train 완료하면 validation data로 성능 평가하고 다음 학습 루프로

이번에는 다른 하나를 validation으로 두고 나머지를 train으로 해서 동일 과정 진행 → K번 반복 → 학습 완료

학습된 모델을 test data로 평가

Dataset Split

Dataset = Train data + Validation data + Test data



학습 진행할 때 Large dataset을 쓰는 경우가 많기 때문에 K-fold cross validation은 시간이 너무 오래 걸림... 따라서 그냥 data 한 번 분할한 걸 그냥 그대로 가져가는 경우가 많음 (Case 2)

Hyperparameter Tuning

Hyperparameter Tuning 예시 1: GridSearch

각 하이퍼 파라미터 별로 후보 값들을 설정해두고, 모델이 학습할 때 해당 후보들 조합을 모두 사용하여 성능을 비교 → 가장 최적의 하이퍼 파라미터 후보를 찾아줌

```python
from sklearn.model_selection import GridSearchCV

# setting params
xgb_params = {
    'n_estimators': [50, 100, 150, 200],
    'max_depth': [1, 3, 5, 7],
    'learning_rate': [0.025, 0.05, 0.075, 0.1],
    'reg_lambda': [0.1, 0.5, 1, 2]
}

# grid search
grid_tree = GridSearchCV(xgb, param_grid=xgb_params,
                         cv=5, refit=True, verbose=1)
grid_tree.fit(X_train, y_train)

print(f"Best parameters: {grid_tree.best_params_}")
print(f"Best cross-validation score: {grid_tree.best_score_}")

# test best case
best_estimator = grid_tree.best_estimator_
y_pred = best_estimator.predict(X_test)
print(f"test accuracy : {accuracy_score(y_test, y_pred)}")
```

하이퍼파라미터 후보 값들 설정

GridSearchCV (k-fold cross validation도 같이)

Best 조합 출력

Best 조합에서 훈련된 모델 불러오기

Hyperparameter Tuning

Hyperparameter Tuning 예시 2: RandomSearch

모든 조합을 다 탐색하는 것이 아닌 샘플링해서 성능 비교

```python
from sklearn.model_selection import RandomizedSearchCV

# setting params
xgb_params = {
    'n_estimators': [50, 100, 150, 200, 250],
    'max_depth': [1, 3, 5, 7, 9],
    'learning_rate': [0.025, 0.05, 0.075, 0.1, 0.125],
    'reg_lambda': [0.1, 0.5, 1, 1.5, 2]
}

# grid search
random_tree = RandomizedSearchCV(xgb, param_distributions=xgb_params,
샘플링되는 하이퍼파라미터 조합의 수를 의미함
                                 cv=5, n_iter=20, refit=True, verbose=1, random_state=42)
fold cross validaion의 k를 의미함
random_tree.fit(X_train, y_train)

print(f"Best parameters: {random_tree.best_params_}")
print(f"Best cross-validation score: {random_tree.best_score_}")

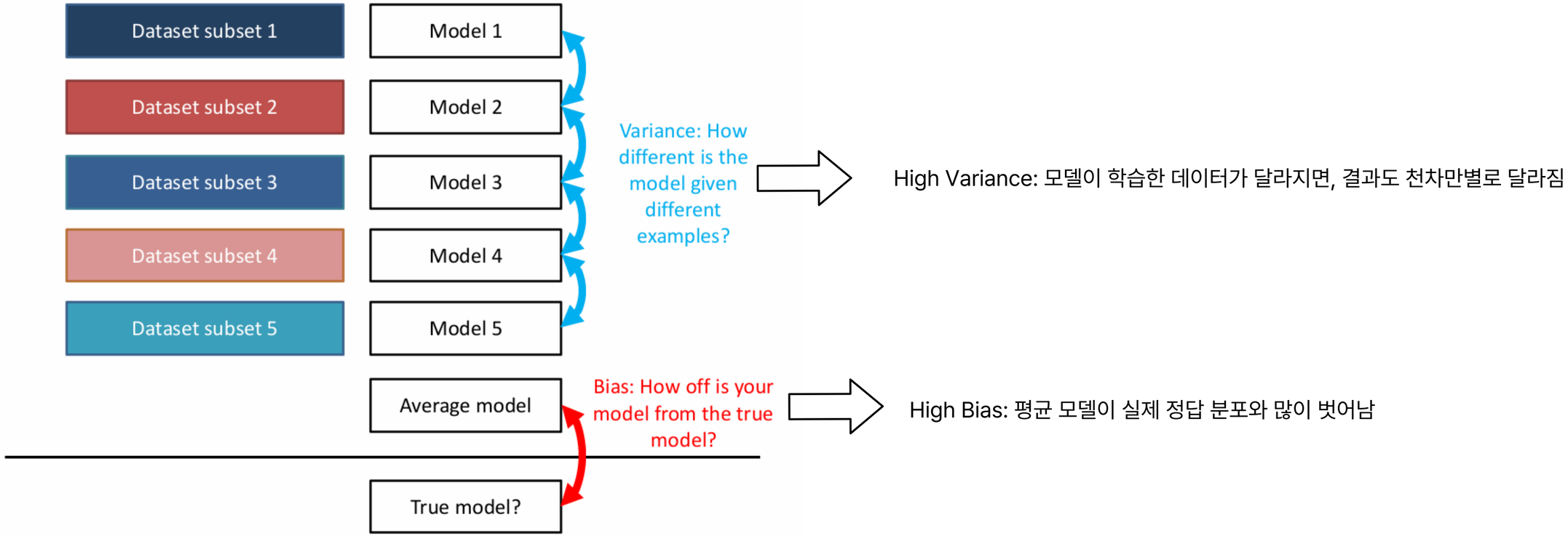
# test best case
best_estimator = random_tree.best_estimator_
y_pred = best_estimator.predict(X_test)
print(f"test accuracy : {accuracy_score(y_test, y_pred)}")
```

RandomizedSearchCV

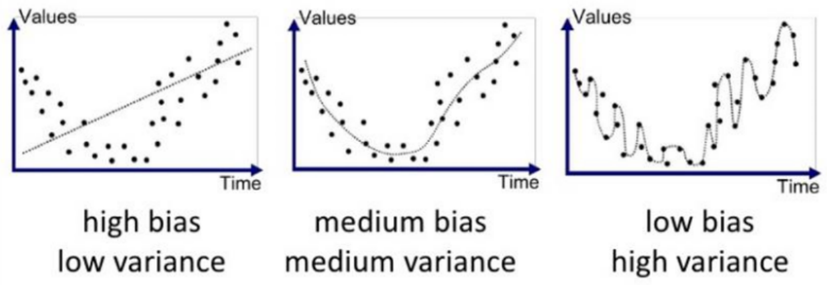
Underfitting & Overfitting

머신러닝이든 딥러닝이든 모델의 성능이 내가 원하는만큼 안 나올 때 이 두 상황 중 어느 상황인건지 판단할 줄 알아야 하고, 그에 맞는 대처법도 알고 있어야 함!!

이 두 상황을 이해하려면, Variance와 Bias의 개념을 알고 있어야 함



Underfitting & Overfitting



Underfitting

Overfitting

Underfitting

모델이 학습 데이터조차 예측을 못함 (학습부터 아예 안된 것)

데이터 양의 문제가 아니라 모델 구조 자체가 task or 데이터와 맞지 않거나, 맞지만 모델이 너무 단순(파라미터 수가 적음)

대처: 모델 구조 자체를 다시 설계하거나 모델을 구성하는 학습 파라미터 수를 늘려 모델을 복잡하게 만들어보기

Overfitting

모델이 훈련 데이터에만 최적화되어 generalization 능력이 떨어짐

(데이터의 general pattern만 파악해야 하는데, 해당 데이터셋만의 노이즈까지 과도하게 학습한 것)

데이터 양에 비해 모델의 구조가 너무 복잡(학습 파라미터 수가 너무 많음 or 학습 파라미터 값이 너무 큼)할 때 주로 발생

대처: 데이터 양을 늘리거나, 모델의 구조를 단순화(파라미터 수를 줄임) 해보기, 오버피팅 방지용 기법들 적용 (ex. 규제, 드롭아웃)

Validation & Test data로 성능 평가 해보고 그래프 그려보기 → overfitting & underfitting 파악

Overfitting 대처: Regularization (규제)

θ = [파라미터 1, 파라미터 2, …., 파라미터 n]

원래 목적 함수(loss)

$$\mathcal{L} = \mathcal{L}_{\text{orig}} + \lambda \|\theta\|_1$$

L1 규제

최소화 해야할 목적 함수(Loss)

목적 함수인 loss에 이 term을 추가해주면 overfitting을 억제할 수 있음

(파라미터 값이 커지면 loss가 커짐: penalty → 파라미터 값을 줄이는 방향으로 학습)

$$\mathcal{L} = \mathcal{L}_{\text{orig}} + \lambda \|\theta\|_2^2$$

L2 규제

- L1 norm

$$\|\boldsymbol{x}\|_1 := \sum_{i=1}^{n} |x_i|$$

- L2 norm

$$\|\boldsymbol{x}\|_2 := \sqrt{x_1^2 + \cdots + x_n^2}$$

- L^p norm (for real numbers $p \geq 1$)

$$\|\mathbf{x}\|_p := \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$$

비지도 학습(Unsupervised Learning) 대표: Clustering

정답지 없이 데이터만 좌표 공간에 던져주면, 가까운 애들끼리 클러스터링 (비지도 학습)

Clustering 목적 함수

이걸 최소화하는 클러스터 중심들과
클러스터에 속한 점들을 찾아야 함

$$J = \sum_{i=1}^{m} \sum_{k=1}^{m} w_{ik} \|x_i - c_k\|^2$$

W: 클러스터에 속해 있지 않은 점이면 0

x: 데이터 포인트

Ck: 클러스터 중심 (데이터로부터 계산, 따로 제공한 정답지가 아님)

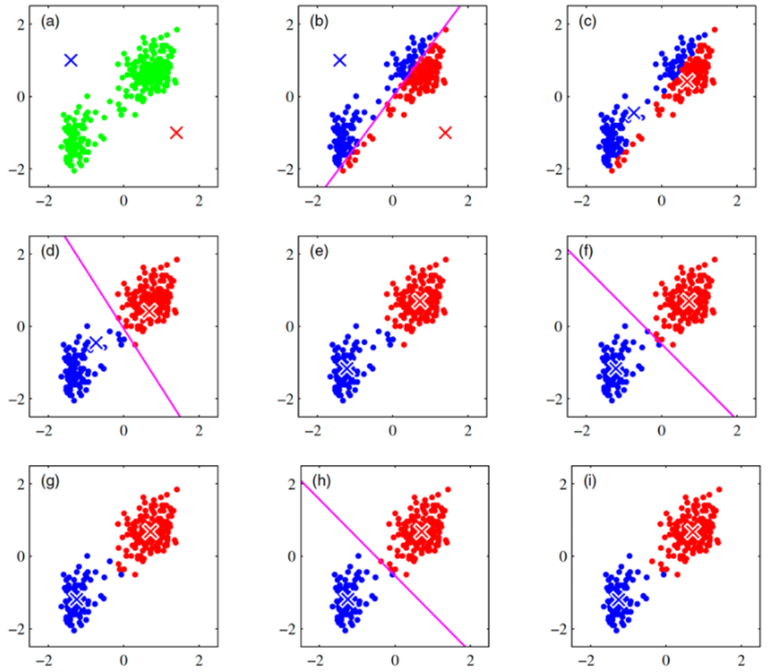
$$w_{ik} = 0 \; if \; the \; data \; point \; does \; not \; belong \; to \; the \; cluster$$
$$w_{ik} = 1 \; if \; the \; data \; point \; belongs \; to \; the \; cluster$$

비지도 학습(Unsupervised Learning) 대표: Clustering

Clustering 알고리즘은 다양

K-means (가장 많이 쓰임)
K-medoids
DBSCAN (특정 case에서 종종 쓰임)
....

K-means Clustering



1. 클러스터 중심 개수 설정 (K: 하이퍼 파라미터)
2. 클러스터 중심 K개 랜덤 초기화
3. 각 점들은 가까운 클러스터 중심에 해당하는 클러스터로 할당
4. 클러스터에 속한 점들의 평균 점을 새로운 클러스터 중심으로 설정
5. 각 점들은 가까운 클러스터 중심에 해당하는 클러스터로 할당

평균해서 중심 계산 → 각 점 클러스터 할당

이 과정을 반복

비지도 학습(Unsupervised Learning) 대표: Clustering

K-means Clustering

https://shabal.in/visuals/kmeans/2.html

1주차 Machine Learning Summary

머신러닝 vs 딥러닝
학습이란? (목적 함수)
지도학습 vs 비지도학습
→ AI Introduction & 기초 개념

Regression 머신러닝 모델
Classification 머신러닝 모델
→ Regression & Classification task 이해 및 머신러닝 학습 전략 case study

결정트리 + GBM (머신러닝 SOTA)
→ 실질적으로 많이 사용하는 머신러닝 모델

Hyperparameter
Data split
Underfitting & Overfitting
→ 학습 기초 이론 및 전략

Clustering
→ 비지도 학습 예